# SAT-Driven Exact Treewidth via Safe Separators and Structure-Aware Encodings

Yannik Krone*
École Polytechnique Fédérale de Lausanne (EPFL)
Lausanne, Switzerland

Sepehr Elahi
École Polytechnique Fédérale de Lausanne (EPFL)
Lausanne, Switzerland

## Abstract

Treewidth is a standard measure of graph structure, but computing it exactly is NP-hard. SAT- and MaxSAT-based approaches provide a uniform way to solve treewidth by encoding it in conjunctive normal form and delegating the search to modern Boolean solvers, thus directly benefiting from continued progress in SAT technology. This report studies how to strengthen such SAT-driven solvers with structural reasoning and low-overhead guidance, both at the meta-algorithmic level and within the encoding itself. Starting from a baseline SAT/MaxSAT formulation, we investigate structure-based simplifications that reduce solver effort while preserving exactness, and we evaluate practical heuristic enhancements (fast upper/lower bounds and solver guidance) within iterative SAT and MaxSAT workflows. Experiments show consistent speedups on instances with exploitable structure, and clarify where SAT-based approaches remain limited.

## Keywords

Treewidth, Exact algorithms, SAT, MaxSAT, Safe Separators, Graph Decomposition, Heuristics

## 1 Introduction

Treewidth is a central structural graph parameter that captures how close a graph is to a tree and, more importantly, when dynamic programming becomes feasible on otherwise intractable problems. A wide range of NP-hard tasks become tractable on graphs of bounded treewidth because the combinatorial explosion can be confined to the width of a tree decomposition rather than the total instance size [11, 16]. The main issue is that computing treewidth exactly is itself NP-hard, so practical treewidth solvers must balance generality against scalability. The literature offers a broad range of tools to tackle the problem of treewidth, from elimination-based algorithms and branch-and-bound search to dynamic programming over decompositions and structural reductions [11, 13, 14]. These approaches are often complementary: heuristics provide quick certificates, while exact procedures refine them to optimality when needed.

It is also known that treewidth can be characterized via a vertex (elimination) ordering to which a corresponding tree-decomposition can be associated, and that among all vertex orderings the optimal treewidth is attained by at least one ordering [11, 16]. Building upon this characterization, SAT- and MaxSAT-based methods offer a uniform, solver-driven route to exact treewidth computation. Instead of engineering dedicated branching rules over elimination orderings, one can encode the existence of an ordering of width at most $w$ into a conjunctive normal form (CNF) and delegate the search to a modern SAT solver.

Berg and Järvisalo [9] developed and evaluated an effective formulation combining an ordering/triangulation encoding with incremental cardinality constraints, supporting iterative SAT and a MaxSAT optimization variant. This formulation cleanly separates modeling from search, but raises a natural question: How can SAT-based treewidth solving be improved in practice by leveraging structural knowledge of the graph more effectively?

This report studies SAT-driven exact treewidth computation through the lens of decomposition. We take the SAT/MaxSAT framework of Berg and Järvisalo as a baseline and investigate how far it can be pushed by integrating safe-separator reasoning. Safe separators, introduced by Bodlaender and Koster [12], support provably sound decomposition rules: splitting a graph along such a separator reduces treewidth computation to independent subinstances that can be solved separately and recombined without changing the optimum. We develop two complementary integrations of safe separators. The first is algorithmic: we wrap the baseline SAT/MaxSAT solver in a recursive procedure that decomposes along safe separators and invokes SAT/MaxSAT only on residual subgraphs, merging the resulting certificates into a global tree-decomposition. The second is encoding-level: we extract a safe-separator decomposition once and flatten it into a single SAT instance by encoding all leaf subgraphs and linking their width constraints globally. These two perspectives expose a trade-off: recursion exploits decomposition opportunities locally, while flattening reduces repeated solver overhead by letting one incremental solve handle all leaves. Beyond decomposition, we also consider lightweight SAT-layer enhancements that preserve exactness. First, inexpensive polynomial-time upper and lower bounds restrict the width region that must be explored, avoiding provably irrelevant SAT/MaxSAT queries in iterative schemes. Second, heuristic elimination orderings obtained while computing an upper bound can be translated into guidance for the SAT solver, biasing the search toward solutions that resemble good heuristic certificates without adding constraints. We evaluate these enhancements both in isolation and end-to-end in combination with separator-based reasoning.

Overall, the report makes four contributions. First, it provides an implementation and empirical baseline for SAT/MaxSAT treewidth solving following the iterative encoding of Berg and Järvisalo [9]. Second, it extends this baseline with inexpensive heuristic bounding and low-overhead solver guidance to reduce the number and difficulty of SAT/MaxSAT queries. Third, it leverages the enhanced

*This work was carried out in the Information and Network Dynamics (INDY) group at EPFL and supervised by Sepehr Elahi. The INDY group is led by Prof. Matthias Grossglauser and Prof. Patrick Thiran.

SAT/MaxSAT core to exploit separator structure in two complementary ways: a recursive meta-solver that decomposes the instance and solves residual subgraphs exactly, and a separator-aware iterative encoding that compiles the decomposition into a single incremental SAT/MaxSAT instance. Fourth, it empirically evaluates these techniques, both individually and in combination, across diverse benchmark families, clarifying when they yield substantial gains and when the remaining bottleneck lies in the baseline SAT/MaxSAT encoding.

## 2 Preliminaries

We briefly recall some notions and notation used throughout the report. The definitions in this section serve as a reference for later sections. Throughout this paper, all graphs are undirected.

### 2.1 Treewidth

**Definition 1:** Treewidth and tree-decompositions
A tree-decomposition of a graph $G = (V, E)$ is a pair $(T, \{X_t\}_{t \in V(T)})$, where $T$ is a tree and for each node $t$ of $T$ we have a bag $X_t \subseteq V$, such that:

(1) Vertex coverage: $\bigcup_{t \in V(T)} X_t = V$.
(2) Edge coverage: For every edge $\{u, v\} \in E$ there is some $t \in V(T)$ with $\{u, v\} \subseteq X_t$.
(3) Running intersection: For every vertex $v \in V$, the set $\{ t \in V(T) \mid v \in X_t \}$ induces a connected subtree of $T$.

The width of a tree-decomposition is $\max_{t \in V(T)} |X_t| - 1$. The *treewidth* of $G$ is the minimum width over all tree-decompositions of $G$, that is, $\mathrm{tw}(G) = \min_{(T, \{X_t\})} \mathrm{width}(T, \{X_t\})$.

**Definition 2:** Treewidth via linear orderings
Fix a linear ordering of the vertices, $v_1 \prec v_2 \prec \cdots \prec v_n$. Given an (undirected) edge relation $E' \subseteq \binom{V}{2}$, call $v_i$ a *predecessor* of $v_j$ (under $\prec$) if $v_i \prec v_j$ and $\{v_i, v_j\} \in E'$, and call $v_j$ a *successor* of $v_i$ in the same situation.

Starting from $E^{(0)} := E$, repeatedly apply the following rule until saturation: whenever two vertices $v_i$ and $v_j$ have a common predecessor $v_k$ (i.e., $v_k \prec v_i$ and $v_k \prec v_j$) and both edges $\{v_k, v_i\}$ and $\{v_k, v_j\}$ are present in the current edge set $E^{(t)}$, add the edge $\{v_i, v_j\}$ if it is not already present. Let $\Delta(E, \prec)$ be the resulting saturated edge set, and define the induced triangulation as

$$\Delta(G, \prec) \;:=\; (V, \Delta(E, \prec)).$$

Orient the edges of $\Delta(G, \prec)$ according to $\prec$:

$$\tilde{\Delta}(E, \prec) \;:=\; \{ (v_i, v_j) \mid \{v_i, v_j\} \in \Delta(E, \prec),\; v_i \prec v_j \}.$$

The *width* of the ordering $\prec$ is the maximum number of successors of any vertex in this ordered triangulation,

$$\mathrm{width}(\prec) \;:=\; \max_{v_i \in V} \left| \{ (v_i, v_j) \in \tilde{\Delta}(E, \prec) \} \right|.$$

Optimal treewidth can equivalently be characterized by

$$\mathrm{tw}(G) \;=\; \min_{\prec} \mathrm{width}(\prec),$$

that is, $\mathrm{tw}(G)$ is the smallest $w$ for which there exists an ordering $\prec$ with $\mathrm{width}(\prec) \leq w$ [9, 11, 16].

**Equivalent elimination and decomposition view.** (Intuition) The triangulation rule above is equivalent to the standard elimination process: vertices are eliminated in the order $v_1, v_2, \ldots, v_n$; when eliminating $v_i$, all of its currently remaining neighbors are connected to each other (introducing fill edges), and $v_i$ is then "removed". The final saturated graph obtained in this way is exactly $\Delta(G, \prec)$, and the successors of $v_i$ in $\tilde{\Delta}(E, \prec)$ are precisely the neighbors that are still present when $v_i$ is eliminated. Hence, $\mathrm{width}(\prec)$ is the largest neighborhood size that ever needs to be cliquified. Furthermore, an elimination order yields a tree-decomposition directly. At the moment $v_i$ is eliminated, form a bag $X_i := \{v_i\} \cup N_i$, where $N_i$ is the set of its current neighbors. Then $|X_i| - 1 = |N_i|$, so the maximum bag size minus one equals $\mathrm{width}(\prec)$. By linking each bag $X_i$ to a later bag that contains $N_i$ (via the earliest vertex of $N_i$ under $\prec$), the bags can be connected into a tree that satisfies the running-intersection property. Therefore, minimizing ordering width is equivalent to minimizing tree-decomposition width [11, 16].

### 2.2 SAT and MaxSAT

**Definition 3:** SAT in CNF
A CNF formula is a conjunction of clauses, e.g., $F = C_1 \wedge \cdots \wedge C_m$. Each clause is a disjunction of literals, e.g., $C = (x_1 \vee \neg x_2 \vee x_5)$. A truth assignment is a map $\pi$ that assigns each Boolean variable $x$ a value in $\{0, 1\}$. The SAT problem asks whether there exists an assignment $\pi$ such that all clauses in $F$ evaluate to true under $\pi$. [10]

**Definition 4:** MaxSAT
In (partial) MaxSAT, the clauses are split into hard clauses $F_h$ and soft clauses $F_s$. The task is to find an assignment $\pi$ that satisfies all clauses in $F_h$ and maximizes the number of satisfied clauses in $F_s$. In this report, the soft clauses are unweighted (all soft clauses have equal weight). [20]

### 2.3 Safe Separators

**Definition 5:** (Safe separator) A *separator* of a graph $G = (V, E)$ is a vertex set $S \subseteq V$ such that $G \setminus S$ has at least two connected components. A separator $S$ is *safe for treewidth* if making $S$ a clique does not increase treewidth [12]; writing $K(S)$ as the clique generated by $S$, this means $\mathrm{tw}(G) = \mathrm{tw}(G \cup K(S))$.

Let $C_1, \ldots, C_r$ be the connected components of $G \setminus S$, and define the separator-augmented subgraphs $G'_i := G[C_i \cup S] \cup K(S)$ for $i = 1, \ldots, r$. Then treewidth decomposes as

$$\mathrm{tw}(G) \;=\; \max\Big( |S| - 1,\; \max_{i=1}^{r} \mathrm{tw}(G'_i) \Big).$$

Since $K(S) \subseteq G'_i$ implies $\mathrm{tw}(G'_i) \geq |S| - 1$ for all $i$, an equivalent shorthand is $\mathrm{tw}(G) = \max_i \mathrm{tw}(G'_i)$. We use these two forms interchangeably throughout the report.

### 2.4 Lower bounds

**Definition 6:** (Minor) A graph $H$ is a *minor* of a graph $G$ if $H$ can be obtained from $G$ by a sequence of vertex deletions, edge deletions, and edge contractions.

Treewidth is *minor-monotone* [14, 21] : if $H$ is a minor of $G$, then

$$H \text{ is a minor of } G \implies \mathrm{tw}(H) \le \mathrm{tw}(G)$$

.

## 2.5 Upper bounds

**Definition 7:** (Elimination width) Let $\prec$ be a linear ordering of $V$. Eliminating vertices in this order means that, for each vertex, we make its current neighborhood a clique (add fill edges) and then remove it. The *elimination width* of $\prec$ is

$$w(\prec) \;=\; \max_{v \in V} \deg_{\prec}(v),$$

where $\deg_{\prec}(v)$ denotes the degree of $v$ at the moment it is eliminated. Any ordering $\prec$ yields an upper bound $\mathrm{tw}(G) \le w(\prec)$, and as mentioned, there exists an ordering attaining equality [13, 16].

# 3 Baseline: SAT-based Treewidth Computation

This section provides a concise overview of the SAT-based framework for exact treewidth computation introduced by Berg and Järvisalo [9], focusing on the components relevant to our subsequent extensions. SAT concerns the satisfiability of CNF formulas (Definition 3). In this setting, for an undirected graph $G = (V, E)$, Berg and Järvisalo encode the decision problem $\mathrm{tw}(G) \le w$ as a CNF formula $\mathcal{F}(G, w)$ such that $\mathcal{F}(G, w)$ is satisfiable if and only if $G$ admits an elimination ordering of width at most $w$ (following Definition 2).

## 3.1 Base encoding

Let $G = (V, E)$ be an undirected graph with $V = \{v_1, \ldots, v_N\}$. The base encoding captures (i) a linear ordering $\prec$ of $V$ and (ii) the ordered triangulation induced by $\prec$. While Berg and Järvisalo's encoding builds on an earlier base encoding [15], in this report, we follow their refined variant. We define the following variables:

**Variable 1: Linear Ordering.** For every pair $1 \le i < j \le N$, introduce a Boolean variable $ord_{ij}$, where $ord_{ij} = 1$ encodes $v_i \prec v_j$. Following Berg and Järvisalo [9], we use the derived literal $ord^*_{ij}$ defined by $ord^*_{ij} = ord_{ij}$ if $i < j$ and $ord^*_{ij} = \neg ord_{ji}$ if $i > j$, so that $ord^*_{ij} = 1$ always means $v_i \prec v_j$ for all $i \ne j$.

**Variable 2: Triangulated Graph Edges.** For all distinct $i, j \in \{1, \ldots, N\}$, introduce a Boolean variable $O_{ij}$. Intuitively, $O_{ij} = 1$ means that the ordered triangulated graph induced by $\prec$ contains the directed edge $(v_i, v_j)$, i.e., $v_j$ is a successor of $v_i$.

**Encoding 1: Base constraints $F_{\mathbf{base}}(G)$.** The CNF formula $F_{\mathrm{base}}(G)$ is the conjunction of clause families that enforce the intended meaning of $ord_{ij}$ and $O_{ij}$. First, $ord^*$ is constrained to form a transitive total order; in particular, for all distinct $i, j, k$, $(ord^*_{ij} \wedge ord^*_{jk}) \to ord^*_{ik}$ is enforced. Second, every original edge must occur in the ordered triangulation in one direction: for each $\{v_i, v_j\} \in E$, add $O_{ij} \vee O_{ji}$. Third, fill edges are introduced whenever two vertices share a common predecessor in the ordered triangulation: for all distinct $i, j, k$, enforce $(O_{ki} \wedge O_{kj}) \to (O_{ij} \vee O_{ji})$. Fourth, orientations are made consistent with the order: for all $i \ne j$, enforce $ord^*_{ij} \to \neg O_{ji}$. Finally, an antisymmetry constraint is added for all $i \ne j$: $\neg O_{ij} \vee \neg O_{ji}$.

## 3.2 Full iterative encoding

Recall from Definition 2 that the width of an ordering is the maximum, over all vertices, of the number of outgoing edges in the ordered triangulated graph. The base CNF $F_{\mathrm{base}}(G)$ already encodes the following constraints: the variables $ord_{ij}$ represent a linear ordering $\prec$, and the variables $O_{ij}$ represent the ordered triangulated graph induced by $\prec$. Enforcing a width bound $w$ means requiring that no vertex has more than $w$ outgoing $O$-edges in this ordered triangulation. Using the $O_{ij}$ variables, this becomes a set of cardinality constraints of the form $C_w(i) : \sum_{j \ne i} O_{ij} \le w$ for all $i \in \{1, \ldots, N\}$.

**Using cardinality networks.** The encoding from Berg and Järvisalo in [9] implements these constraints with cardinality networks [4, 5], so that many values of $w$ can be tested incrementally using the same CNF. Fix $i \in \{1, \ldots, N\}$ and define the set of out-edge literals $A_i = \{\, O_{ij} \mid j \ne i \,\}$. A cardinality network encoding $C(A_i)$ introduces auxiliary variables $y^i_1, \ldots, y^i_{N-1}$ with the following threshold meaning: for any assignment $\pi$ and any $k$, if $\pi(y^i_k) = 0$ then at most $k - 1$ literals in $A_i$ are true under $\pi$. For example, $\pi(y^i_3) = 0$ implies that among the literals in $A_i$ (at most) two are true, i.e., $v_i$ has at most two successors in the ordered triangulation. Therefore, the width constraint $C_w(i)$ is enforced by the single unit clause $\neg y^i_{w+1}$ on top of $C(A_i)$: if $\pi(\neg y^i_{w+1}) = 1$, then at most $w$ literals in $A_i$ can be true, which is exactly $\sum_{j \ne i} O_{ij} \le w$.

**Variable 3: Width-threshold.** To reuse the same CNF for many values of $w$, introduce variables $W_0, \ldots, W_{N-2}$ with $W_w \leftrightarrow (\neg y^1_{w+1} \wedge \neg y^2_{w+1} \wedge \cdots \wedge \neg y^N_{w+1})$ for each $w \in \{0, \ldots, N-2\}$. Intuitively, $W_w = 1$ means that all vertices satisfy the bound "out-degree $\le w$". Monotonicity is captured by adding $W_w \to W_{w+1}$ for all $w \in \{0, \ldots, N-3\}$.

**Encoding 2: Berg–Järvisalo iterative encoding $F_{\mathbf{iter}}(G)$.** The full iterative CNF is

$$F_{\mathrm{iter}}(G) \;=\; \underbrace{F_{\mathrm{base}}(G)}_{\text{base (order + triangulation)}} \;\wedge\; \underbrace{\Big(\bigwedge_{i=1}^{N} C(A_i)\Big)}_{\substack{\text{cardinality networks}\\ \text{on } A_i = \{O_{ij} \mid j \ne i\}}} \;\wedge\; \underbrace{\Big(\bigwedge_{w=0}^{N-2} \Big(W_w \leftrightarrow \bigwedge_{i=1}^{N} \neg y^i_{w+1}\Big)\Big)}_{\text{width-threshold}} \;\wedge\; \underbrace{\Big(\bigwedge_{w=0}^{N-3} (W_w \to W_{w+1})\Big)}_{\substack{\text{monotonicity}\\ \text{in } w}}.$$

## 3.3 Solving for the exact treewidth and tree-decomposition

The iterative CNF $F_{\mathrm{iter}}(G)$ (Encoding 2) is built so that different width bounds can be checked by changing only the assumption on the width-threshold variables $W_w$. For any $w \in \{0, \ldots, N-2\}$, define the width-$w$ decision query as

$$F(G, w) \;:=\; F_{\mathrm{iter}}(G) \;\wedge\; W_w.$$

Equivalently, a SAT solver is called on the fixed CNF $F_{\mathrm{iter}}(G)$ under the single assumption literal $W_w = 1$. By construction, $F(G, w)$ is satisfiable if and only if $\mathrm{tw}(G) \le w$ [9].

**Iterative SAT (search over $w$).** Since the property $\mathrm{tw}(G) \le w$ is monotone in $w$, Berg and Järvisalo [9] recover the exact treewidth by probing values of $w$ until the minimum satisfiable bound is identified. In linear upward search (*iter-lin-up*), queries start at $w = 1$

and increase by one until the first satisfiable query $F(G, w)$ is found. In linear downward search (*iter-lin-down*), queries start at $w = N-2$ and decrease by one while satisfiable; the first unsatisfiable query identifies that the previous width is optimal. As a corner case, if already $F(G, N-2)$ is unsatisfiable, then no ordering has width at most $N-2$ and $\mathrm{tw}(G) = N-1$. In binary search (*iter-bin*), a current interval $[L, U]$ known to contain $\mathrm{tw}(G)$ is maintained, the midpoint width is tested, and the interval is updated depending on satisfiability.

**MaxSAT.** Instead of probing widths iteratively using a SAT solver, the encoding can also be instantiated as a single partial MaxSAT optimization problem that selects the smallest feasible width in one call [9]. In MaxSAT optimization (*maxsat*) with equal weights, the instance is

$$\left( \underbrace{F_{\mathrm{iter}}(G)}_{\text{hard}},\ \underbrace{\{\, (W_w, 1) \mid w = 0, \ldots, N-2 \,\}}_{\text{soft unit clauses, weight 1}} \right),$$

that is, all clauses of $F_{\mathrm{iter}}(G)$ are hard and each $W_w$ is added as a soft unit clause with weight 1. Because $W_w \to W_{w+1}$ holds for all $w$, any assignment satisfying the hard clauses exhibits a threshold behavior: there exists an index $t$ such that $W_0, \ldots, W_{t-1}$ are false and $W_t, \ldots, W_{N-2}$ are true. Maximizing the number of satisfied soft clauses therefore minimizes this threshold index $t$, which corresponds exactly to the treewidth $\mathrm{tw}(G)$.

**Recovering a tree-decomposition.** Whenever a SAT call returns a satisfying assignment for some optimal width bound $w$ (or MaxSAT returns an optimal model), we decode the model into an explicit tree-decomposition: we read off an elimination ordering from the $ord_{ij}$ variables and the triangulation (fill) edges from the $O_{ij}$ variables, and convert this information into a tree-decomposition. This decoding step runs in polynomial time, more details for the implementation is given in Appendix F.5.

**SAT/MaxSAT backend.** In the remainder of this report, the solver modes *iter-lin-up*, *iter-lin-down*, *iter-bin*, and *maxsat*, all based on the standard Berg–Järvisalo encoding, are collectively referred to as the *(atomic) SAT/MaxSAT backend* or the *regular solver*. These exact solver modes serve as our baseline and are invoked as a backend inside HybridSolver (Section 5) whenever no further safe-separator decomposition or additional pruning is available.

## 4 Safe-Separator Decomposition

We now instantiate the general notion of safe separators (Definition 5) with concrete separators. All separators used in this work are safe, so decomposition preserves optimal treewidth: after splitting along a safe separator $S$, the resulting subinstances can be solved independently and recombined. These concrete separators are used later in two places: first as the main decomposition mechanism inside HybridSolver (Section 5), and secondly in our separator-aware iterative encodings (Section 6).

**Disconnected components.** If $G$ has connected components $G_1, \ldots, G_k$, then $S = \varnothing$ is a safe separator and Definition 5 yields

$$\mathrm{tw}(G) = \max_{j \in \{1,\ldots,k\}} \mathrm{tw}(G_j).$$

Hence, treewidth can be computed independently on each connected component and combined by taking the maximum. See Section F.1 for more details.

**Articulation points.** An articulation point is a vertex $v$ whose removal disconnects the graph, i.e., $S = \{v\}$ is a separator. Since singleton sets are cliques, turning $S$ into a clique does not change anything, and $S$ is safe by Definition 5. Implementation details on how to detect articulation points are in Section F.2.

**Clique separators.** If a separator $S$ induces a clique, then $G \cup K(S) = G$, so $S$ is safe by Definition 5. In our implementation, clique separators are detected by enumerating cliques up to a small fixed size bound and checking whether $G \setminus S$ disconnects the graph; see Section F.3.

**Almost-clique separators.** Almost-clique separators extend clique separators by allowing $S$ to become a clique after removing a a vertex. Under the sufficient conditions identified by Bodlaender and Koster [12], minimal almost-clique separators are safe for treewidth, so they can be used with the same decomposition-and-recombine scheme as above. In practice, we generate candidate separators using Tamaki's heuristic for listing almost-clique minimal separators [23], and then enforce the required safety conditions.

## 5 HybridSolver

This section introduces our HybridSolver (also later referred to as *hybrid*), an exact meta-solver that integrates the safe-separator machinery of Section 4 with the SAT/MaxSAT backends from Section 3. The construction is presented in modular building blocks, which are then combined into the final recursive solver.

### 5.1 Goal and interface

Given a graph $G = (V, E)$, HybridSolver returns the exact treewidth $\mathrm{tw}(G)$ together with a tree-decomposition witnessing this width. Our goal is to spend as much time as possible in inexpensive (polynomial-time) structure and bounds, and to call the atomic SAT/MaxSAT backend only on residual subinstances where decomposition cannot proceed further.

### 5.2 Building block I: heuristic bounds

At every recursion node, on a subgraph $H = (V_H, E_H)$, we compute a lower bound $LB(H)$ and an upper bound $UB(H)$ of the treewidth.

**Lower bound $LB(H)$.** We compute the lower bound using the Minor-Min-Width (MMW) heuristic (see Algorithm 4 for more details) from Bodlaender and Koster [14] which repeatedly contracts edges to generate a sequence of minors (Definition 6) and updates the bound with the degree of the contracted vertex. By minor-monotonicity, this yields $LB(H) \leq \mathrm{tw}(H)$. The procedure is polynomial-time.

**Upper bound $UB(H)$.** We compute the upper bound from a good candidate (greedy) elimination ordering $\prec$ (Algorithm 5), following Bodlaender and Koster [13]. Eliminating vertices along $\prec$ yields an elimination width $w(\prec)$, and therefore $\mathrm{tw}(H) \leq UB(H) := w(\prec)$ (Definition 7). We also convert the elimination trace into a tree-decomposition $\mathcal{D}_{UB}$ of width $UB(H)$ using the standard ordering-to-decomposition construction (See implementation details F.5). This procedure runs in polynomial time.

**Immediate pruning when bounds meet.** If $LB(H) = UB(H)$, then $\mathrm{tw}(H)$ is certified and the recursion terminates on $H$ without invoking SAT/MaxSAT. This is the cheapest (and often most effective) early-exit condition. We then return the tree-decomposition stemming from the upper-bound.

## 5.3 Building block II: safe-separators

If bounds do not meet, we attempt to split the instance using safe separators from Section 4.

**Recursion on a safe separator.** Let $S$ be a found safe separator and let $C_1, \ldots, C_r$ be the connected components of $H \setminus S$. We recurse on the separator-augmented subgraphs $H_i := H[C_i \cup S] \cup K(S)$, and recombine by Definition 5:

$$\mathrm{tw}(H) \;=\; \max\Big(|S| - 1, \max_{i=1}^{r} \mathrm{tw}(H_i)\Big).$$

We recursively merge returned tree-decompositions through a hub bag containing $S$ (Section F.9).

**Heuristic to pick separators.** When multiple candidate separators exist (notably for clique / almost-clique enumeration), we propose to rank candidates using the scoring heuristic of Section F.10, favoring separators that minimize the size of the largest resulting component.

## 5.4 Building block III: bound propagation through decomposition

HybridSolver maintains bounds consistently across recursion, so that solving the subinstances yields a tighter interval for the parent. Accordingly, once all children $H_i$ of a safe-separator split with separator $S$ have been processed, we update the bounds of $H$ by

$$\begin{aligned} LB(H) \;&\leftarrow\; \max\Big(LB(H),\ |S| - 1,\ \max_i LB(H_i)\Big), \\ UB(H) \;&\leftarrow\; \min\Big(UB(H),\ \max\big(|S| - 1, \max_i UB(H_i)\big)\Big). \end{aligned}$$

## 5.5 Building block IV: SAT/MaxSAT backend with optimizations

If no safe separator is found, we fall back to the SAT/MaxSAT backend (Section 3). Crucially, it does not call the backend "cold": it reuses the heuristic information computed at the current recursion node—a certified interval $[LB(H), UB(H)]$ and a good elimination ordering—to guide the exact search without compromising correctness.

**SAT/MaxSAT backend interface.** We remind that, on a subinstance $H$, the backend relies on the iterative CNF $F_{\mathrm{iter}}(H)$ (Encoding 2) built from ordering variables $ord_{ij}$ (Variable 1), ordered-triangulation variables $O_{ij}$ (Variable 2), and width thresholds $W_w$ (Variable 3). A width-$w$ query is expressed as

$$F(H, w) \;:=\; F_{\mathrm{iter}}(H) \;\wedge\; W_w,$$

i.e., a SAT call on the fixed CNF $F_{\mathrm{iter}}(H)$ under the single assumption $W_w = 1$. Exact solving is performed either by iterative SAT over $w$ (*iter-lin-up*, *iter-lin-down*, *iter-bin*) or by a single *maxsat* call (Section 3.3). We propose two low-overhead refinements that exploit the available heuristics while preserving exactness.

**Algorithm 1** HybridSolver recursion (simplified)

```
function HybridSolve(G)
    (LB, UB, ≺, D_UB) ← ComputeBounds(G)
    if LB = UB then
        return (UB, D_UB)
    end if
    (S, {C_1, ..., C_r}) ← FindBestSafeSeparator(G)
    if S ≠ ⊥ then
        for i = 1 to r do
            G_i ← G[C_i ∪ S] ∪ K(S)
            (w_i, D_i) ← HybridSolve(G_i)
        end for
        w ← max(|S| − 1, max_i w_i)
        D ← MergeDecompositions((D_i)_i, S)
        return (w, D)
    else
        return AtomicSolve(G, LB, UB, ≺)    ▷ Berg–Järvisalo encoding + bounds-aware + warm-start
    end if
end function
```

**Optimization 1: bounds-aware solving.** Instead of probing widths over the full trivial range, we restrict the backend to the certified interval $[LB(H), UB(H)]$. For iterative SAT, this limits the number of tested widths to at most $\Delta + 1$ for linear scans and $\lceil \log_2(\Delta + 1) \rceil$ for binary search, where $\Delta = UB(H) - LB(H)$. For MaxSAT, we keep $F_{\mathrm{iter}}(H)$ as hard clauses and add unit soft clauses only for thresholds within the interval:

$$\left( \underbrace{F_{\mathrm{iter}}(H)}_{\text{hard}},\ \underbrace{\{\, (W_w, 1) \mid w = LB(H), \ldots, UB(H) \,\}}_{\text{soft unit clauses, weight 1}} \right).$$

The empirical impact of bounds-aware solving is reported in Section 7.4 (Table 1).

**Optimization 2: warm-starting via phase hints on $ord_{ij}$.** The greedy upper bound also provides an elimination ordering $\prec$, which we translate into phase (polarity) hints for the ordering variables $ord_{ij}$ (Variable 1) consistent with $\prec$. These are solver preferences only: no clauses are added, so satisfiability and optimality are unchanged. The observed effect is illustrated in Figure 15 and discussed in Section 7.4; implementation details are given in Section F.8.

## 5.6 Putting the building blocks together: the HybridSolver recursion

We obtain the complete solver by combining (i) bounds, (ii) safe-separator decomposition, (iii) bound propagation, and (iv) an optimized atomic backend that is *bounds-aware* and supports *warm-starting*. See Algorithm 1.

## 5.7 Correctness and complexity

**Correctness.** If $LB = UB$, the value is certified by valid lower and upper bounds. If a safe separator is applied, recursing on the induced subgraphs is correct by Definition 5. If no separator is found,

the atomic backend returns the exact treewidth. Bounds-aware restriction only excludes widths outside $[LB, UB]$, and warm-starting only provides solver hints; neither affects satisfiability or optimality.

**Complexity.** The per-node overhead is polynomial: bounds computation (MMW + greedy elimination), separator detection (linear-time routines plus bounded-size enumeration for clique / almost-clique candidates), and splitting/merging (output-linear in the produced decompositions). The worst-case runtime remains exponential overall because the atomic SAT/MaxSAT call solves an NP-hard problem.

## 6 Separator-Aware Iterative Encoding

This section introduces a separator-aware iterative encoding that compiles a whole safe-separator decomposition into a single incremental SAT/MaxSAT instance. Unlike HybridSolver (Section 5), which recurses and invokes an atomic backend at the leaves, we (i) precompute a safe-separator decomposition tree, (ii) build an independent Berg–Järvisalo iterative encoding for each leaf, and (iii) connect the leaves through a *shared* global width threshold. Iterative SAT (or MaxSAT) then runs on one global formula, reusing learned clauses across all leaves and all tested widths.

### 6.1 Recursion tree and leaf instances

We build a safe-separator recursion tree for the input graph $G$ using the same separator classes as in HybridSolver (Section 4). Whenever a safe separator $S$ is found in a subgraph $H$, we split $H \setminus S$ into connected components and recurse on the corresponding separator-augmented subgraphs $H[C_i \cup S] \cup K(S)$. This process continues until no further safe separator is found (or a recursion limit is reached), resulting in leaf graphs $G_1, \ldots, G_L$. By Definition 5, cliquifying the separator preserves treewidth, and the treewidth of the original graph equals the maximum treewidth among the leaf instances.

### 6.2 Leaf-local iterative encodings

For each leaf $\ell \in \{1, \ldots, L\}$, we build an independent copy of the Berg–Järvisalo iterative encoding (Encoding 2) on $G_\ell$, and we namespace all variables by $\ell$ so that the leaf formulas are syntactically disjoint.

**Variable 4: Leaf-local ordering.** For each leaf $\ell$ and $1 \le i < j \le n_\ell$, introduce $ord^\ell_{ij}$ encoding the local order $v^\ell_i \prec v^\ell_j$, with derived literals $(ord^\ell)^*_{ij}$ as in Variable 1.

**Variable 5: Leaf-local ordered-triangulation edges.** For each leaf $\ell$ and all distinct $i, j \in \{1, \ldots, n_\ell\}$, introduce $O^\ell_{ij}$ as in Variable 2 but restricted to $G_\ell$.

**Variable 6: Leaf-local width-threshold.** For each leaf $\ell$ and each $w \in \{0, \ldots, n_\ell - 2\}$, introduce $W^\ell_w$ with the same incremental meaning as Variable 3 (width $\le w$ for *that leaf*).

**Encoding 3: Leaf-local iterative encoding $F^\ell_{\text{iter}}(G_\ell)$.** For each leaf $\ell$, let $F^\ell_{\text{iter}}(G_\ell)$ be Encoding 2 instantiated on $G_\ell$ using Variables 4, 5, and 6. The leaf part of the global CNF is simply

$$F_{\text{leaves}}(G, T) \;:=\; \bigwedge_{\ell=1}^{L} F^\ell_{\text{iter}}(G_\ell).$$

### 6.3 Global width variables and leaf linking

We now introduce one shared width threshold, and force it to agree with the leaves.

**Variable 7: Global width-threshold.** Let $n := |V(G)|$. Introduce global variables $\widehat{W}_0, \ldots, \widehat{W}_{n-2}$, where $\widehat{W}_w = 1$ is intended to mean "all leaves have width $\le w$". For a fixed $w$, leaves with $n_\ell \le w+1$ are certainly within width $w$ since $\mathrm{tw}(G_\ell) \le n_\ell - 1 \le w$. We therefore only need to constrain the remaining leaves. Define the active-leaf index set

$$\mathcal{L}(w) \;:=\; \{\ell \in \{1, \ldots, L\} \mid n_\ell \ge w + 2\}.$$

We enforce the intended semantics by the equivalence

$$\widehat{W}_w \;\leftrightarrow\; \bigwedge_{\ell \in \mathcal{L}(w)} W^\ell_w.$$

If $\mathcal{L}(w) = \varnothing$, Equation (7) reduces to the unit clause $\widehat{W}_w$.

**Global monotonicity.** As in the baseline iterative encoding, we add $\widehat{W}_w \to \widehat{W}_{w+1}$ for all $w \in \{0, \ldots, n-3\}$.

**Encoding 4: Separator-aware iterative CNF $F_{\text{sep-iter}}(G, T)$.** The full separator-aware formula is

$$F_{\text{sep-iter}}(G, T) \;=\; \underbrace{F_{\text{leaves}}(G, T)}_{\text{leaf iterative encodings}} \;\wedge\; \underbrace{\Big(\bigwedge_{w=0}^{n-3} (\widehat{W}_w \to \widehat{W}_{w+1})\Big)}_{\text{global monotonicity}}$$
$$\wedge\; \underbrace{\Big(\bigwedge_{w=0}^{n-2} \big(\widehat{W}_w \leftrightarrow \bigwedge_{\ell \in \mathcal{L}(w)} W^\ell_w\big)\Big)}_{\text{link global width to all relevant leaves}}.$$

### 6.4 Using the encoding for SAT and MaxSAT

**Iterative SAT.** A width-$w$ decision query is a single SAT call on the fixed CNF $F_{\text{sep-iter}}(G, T)$ under the assumption $\widehat{W}_w = 1$. Monotonicity of $\widehat{W}_w$ supports the same probing strategies as in Section 3.3 (linear up/down or binary search).

**MaxSAT.** We form a partial MaxSAT instance with all clauses of $F_{\text{sep-iter}}(G, T)$ hard and unit soft clauses on $\widehat{W}_w$ :

$$\Bigg(\underbrace{F_{\text{sep-iter}}(G, T)}_{\text{hard}},\; \underbrace{\{\,(\widehat{W}_w, 1) \mid w = 0, \ldots, n-2\,\}}_{\text{soft}}\Bigg).$$

### 6.5 Solver workflow and decomposition reconstruction

Our separator-aware solver proceeds in two phases: (1) build and solve one global incremental SAT/MaxSAT instance, and (2) decode a tree-decomposition by combining leaf tree-decompositions along the separators stored in the recursion tree.

**Data stored in the recursion tree.** The safe-separator recursion produces a rooted tree $T$ whose internal nodes store the separator $S$ used to split the current subgraph and whose edges identify the corresponding separator-augmented children. Each leaf $\ell$ stores its leaf graph $G_\ell$ and a relabeling map $\phi_\ell : V(G_\ell^{\text{local}}) \to V(G)$ that translates the leaf's local vertex ids (back) to the original vertex names (of the input graph).

**Algorithm 2** Separator-aware incremental solving and reconstruction

**Require:** Graph $G$
**Ensure:** Tree-decomposition $\mathcal{D}$ of $G$ of minimum width (via SAT/MaxSAT optimum)

```
Build safe-separator recursion tree T of G; obtain leaves {G_ℓ}_{ℓ=1}^L and maps φ_ℓ
Build global CNF F_sep-iter(G, T) (leaf encodings + global Ŵ + linking/monotonicity)
Find optimum width w* by incremental SAT probes (assumptions Ŵ_w) or MaxSAT.
for ℓ = 1 to L do
    M_ℓ ← projection of M onto leaf-ℓ variable ids (using the CNF namespacing map)
    Extract leaf order π_ℓ from ord^ℓ in M_ℓ; extract fill edges from O^ℓ
    D_ℓ^local ← build a tree decomposition from π_ℓ and O^ℓ on G_ℓ
    D_ℓ ← map bags of D_ℓ^local through φ_ℓ into V(G)
end for
return MergeUp(T, {D_ℓ})        ▷ post-order: add hub bag S at each internal node and connect to children
```

**Decoding leaf solutions.** A satisfying assignment of the global CNF fixes, for each leaf $\ell$, an elimination ordering through the corresponding namespaced ordering variables. For each leaf, we project the model to its local variables, extract an elimination order, and construct a tree-decomposition $\mathcal{D}_\ell$, which is then mapped back to the original vertex set.
The procedure MergeUp traverses the recursion tree bottom-up and combines the leaf decompositions using the stored separators via MergeDecompositions (Algorithm 6), ultimately producing a tree-decomposition of the original graph.

## 6.6 Correctness and complexity

**Correctness.** Let $T$ be the safe-separator recursion tree of $G$ with leaf graphs $G_1, \ldots, G_L$, where each split along a safe separator $S$ recurses on separator-augmented subgraphs $H[C_i \cup S] \cup K(S)$. By the definition of safe separators, cliquifying $S$ preserves treewidth, and the treewidth of any subgraph equals the maximum treewidth of its children. Propagating this property through the recursion yields that $\mathrm{tw}(G)$ is the maximum treewidth among the leaves.

Fix a width bound $w$. For each leaf $\ell$ with at least $w + 2$ vertices, the standard iterative encoding $F_{\text{iter}}^\ell(G_\ell)$ is satisfiable under the assumption $W_w^\ell$ if and only if $\mathrm{tw}(G_\ell) \le w$. Smaller leaves satisfy the bound trivially and need not be encoded at width $w$. The linking constraints enforce

$$\widehat{W}_w \leftrightarrow \bigwedge_{\ell \in \mathcal{L}(w)} W_w^\ell,$$

ensuring that the global width bound $w$ holds simultaneously for all relevant leaves. Consequently, the separator-aware encoding is satisfiable under $\widehat{W}_w$ if and only if $\mathrm{tw}(G) \le w$. Iterating over $w$ therefore identifies the smallest feasible width, which equals $\mathrm{tw}(G)$. The same reasoning applies to the MaxSAT formulation: monotonicity of the global width variables ensures that maximizing the number of satisfied width constraints yields the exact treewidth.

**Complexity.** The size of the separator-aware formula is dominated by the sum of the leaf encodings: we include one full iterative Berg–Järvisalo CNF $F_{\text{iter}}^\ell(G_\ell)$ per leaf, and add only a small global linking layer. This layer consists of (i) a linear number of monotonicity clauses for the global thresholds $\widehat{W}_w$, and (ii) linking clauses that connect each $\widehat{W}_w$ to the active leaves at width $w$. In the worst case, this contributes $O(nL)$ additional clauses, where $n = |V(G)|$ and $L$ is the number of leaves.

The contribution of the flattened separator recursion overhead is polynomial, while the overall worst-case runtime of course remains exponential, as exact treewidth is NP-hard; the goal of the construction is to let one incremental SAT/MaxSAT run share learned information across all leaf subinstances and across all tested widths. Note also that on the reconstruction side, all steps run in polynomial time: each leaf is decoded into a tree-decomposition in polynomial time (Appendix F.5), and the resulting leaf decompositions are merged bottom-up along the recursion tree in polynomial time (Algorithm 6).

## 7 Experimental Evaluation

This section evaluates the solvers introduced in Section 3 (Regular), Section 5 (HybridSolver), and Section 6 (Separator-aware) across several benchmark settings, focusing on runtime, solved instances under time limits, and the practical impact of exploiting safe-separator structure. All experiments were run on a desktop machine with an Intel Core i7-10700K (8C/16T) and 64 GB RAM (CachyOS / Arch Linux).

**General Experimental Setting.** Unless stated otherwise, we compare the three configurations *Regular* which is our baseline, *Hybrid*, and *Separator-aware* throughout. Each configuration is evaluated under four exact modes: `maxsat`, `iter-bin`, `iter-lin-up` and `iter-lin-down`. Unless stated otherwise, results are reported per mode and/or aggregated across modes. All SAT-based runs use Glucose 4 as the underlying SAT solver [6, 7], and MaxSAT runs use RC2 as the optimizer backend (internally calling a SAT solver). Our implementation also supports MiniSat 2.2 [17] and can be extended with minimal effort to other incremental SAT solvers, as solver interaction is isolated behind a thin backend wrapper. We report results on four settings: (1) an instance set closely matching the original Berg–Järvisalo benchmark, with some differences due to unavailable graphs and version discrepancies across sources; (2) pseudo-random clustered graphs; (3) Erdős–Rényi random graphs; and (4) the PACE2017 exact treewidth benchmark. For randomized synthetic benchmarks, we keep track of the random seeds for reproducibility. In addition to end-to-end comparisons, we also evaluate in isolation (i) the two SAT-layer enhancements used by the HybridSolver for atomic backend —*bounds-aware solving* and *warm-start hinting*—and (ii) the usefulness of the polynomial-time heuristic bounds as standalone guidance/pruning signals.

### 7.1 Berg–Järvisalo Benchmark

We first evaluate on the benchmark used by Berg and Järvisalo to assess SAT-based treewidth computation [9]. Following their setup, we consider a 34-graph collection gathered from standard treewidth instance archives and Bayesian–network repositories, including `libtw.zip` [2], `coloring.zip` [3], and the `bnlearn` repository [1]. These instances cover a range of graph structures at moderate sizes and are a widely used reference point for comparing SAT/MaxSAT-based exact treewidth solvers.

**Experimental Setting.** For each instance graph, we run each configuration on all modes (following the general experimental setting 7), with a timeout of 10 minutes per run. Runs exceeding the limit are counted as timeouts.

**Results.** Figure 1 plots the cumulative number of solved instances over time. Both separator-based approaches dominate the Regular baseline and solve more instances without timing out, and this, independently of which underlying mode is used. *Hybrid* consistently reaches higher solved counts earlier and remains the most robust near the timeout, while *Separator-aware* also improves substantially over *Regular*, confirming that exposing safe-separator structure yields practical gains even within a SAT/MaxSAT pipeline. Full raw results, pairwise comparison and aggregated results are provided in Appendix A.

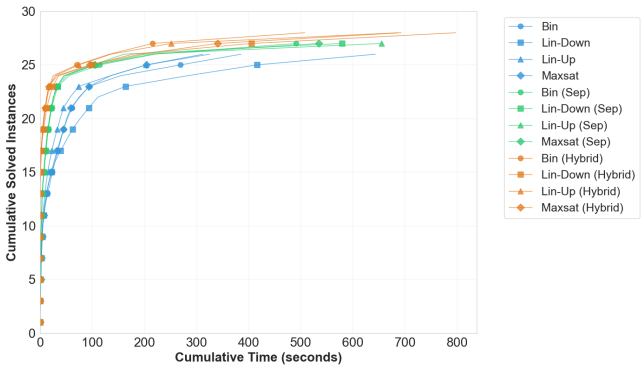


**Figure 1: Cumulative number of solved Berg–Järvisalo benchmark instances over time.**

### 7.2 Pseudo-Random Clustered Graphs

We next evaluate on a synthetic benchmark designed to exhibit strong separator structure. Instances consist of $k$ dense clusters of size $n$ connected by a sparse inter-cluster backbone, so that articulation points and small clique-like separators occur frequently (see Appendix B.1 for more details and representative example instance). Such clustered graphs model systems composed of tightly connected components with sparse interconnections, a structure common in many real-world networks and graphical models.

**Experimental Setting.** We generate instances over a grid of parameters $(k, n)$ using fixed random seeds, where k (numbers of clusters) ranges from 2 to 14 and n (cluster sizes in amounts of vertices) from 5 to 20. Each run is repeated five times per fixed $(k, n)$, and we report mean wall-clock runtime; runs exceeding the timeout of 60 seconds are counted as timeouts. Note that a grid is considered as timed-out if a majority out of the 5 instances times out for given configuration and associated mode.

**Results.** Figure 2 shows runtime heatmaps ($\log_{10}$ seconds) across the $(k, n)$ grid. Both separator-based approaches substantially outperform the *Regular* baseline, confirming that safe separators capture the dominant structure in these clustered instances and shrink the hardest SAT/MaxSAT subproblems. The improvement is often by orders of magnitude: large regions that time out for *Regular* become solvable in seconds for *Hybrid* and *Separator-aware*. Interestingly, *Separator-aware* can even outperform *Hybrid* on the hard right-hand side of the grid: there are parameter regions where *Hybrid* still hits the time limit while *Separator-aware* completes it. A plausible explanation is that the separator-aware encoding exposes more reuse to the SAT/MaxSAT solver. HybridSolver prunes primarily at the algorithmic level by decomposing the instance and tightening bounds, but the SAT work performed in each leaf remains largely isolated across recursive calls. In contrast, the separator-aware approach compiles all leaves into a single *incremental* SAT/MaxSAT instance with a shared global width threshold, so learned clauses and propagations can be reused across leaves and across repeated width probes. As a result, a global width-$w$ query can be refuted as soon as one leaf forces width $> w$, without fully exploring the remaining leaves, which explains why separator-aware can outperform *Hybrid* on the hardest region of the grid.

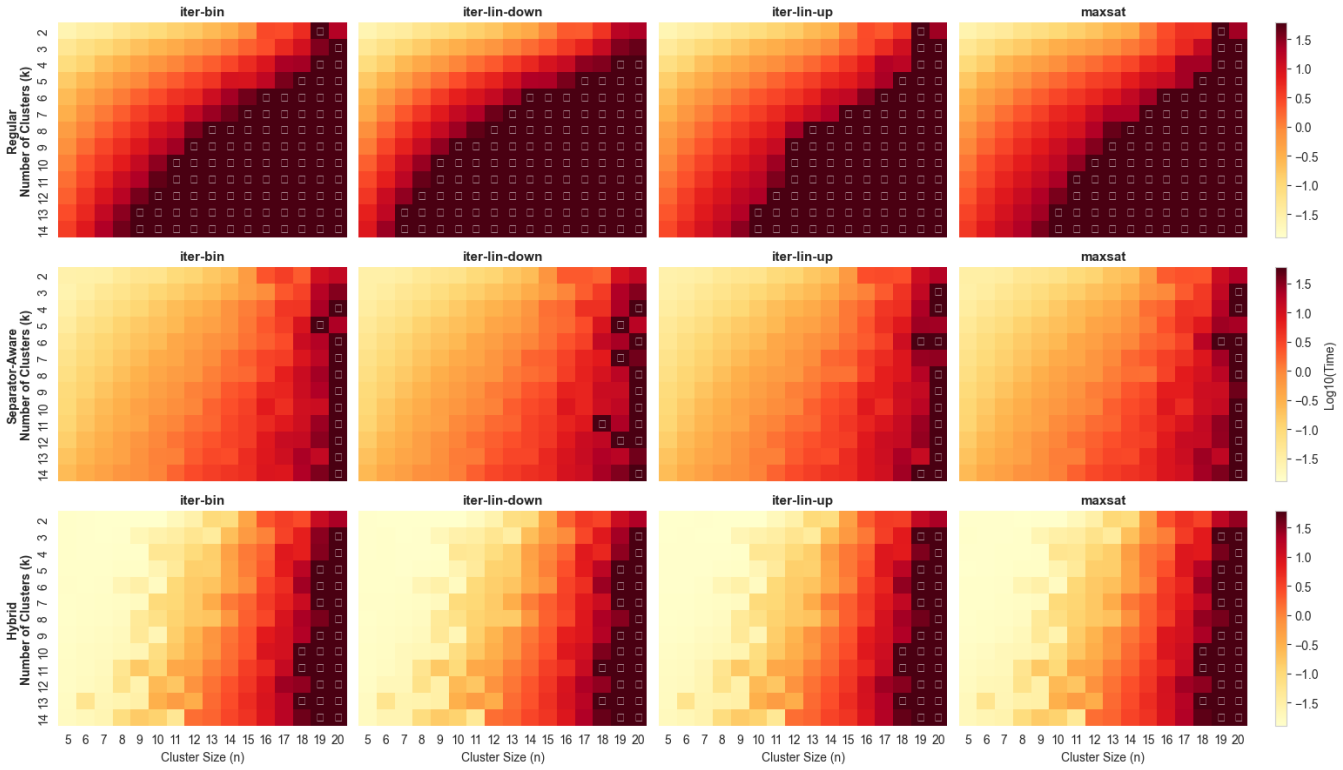


**Figure 2: Pseudo-random clustered benchmark: runtime heatmaps ($\log_{10}$ seconds) over cluster size $n$ and number of clusters $k$. White markers denote timeouts.**

## 7.3 Random Graphs

We next evaluate on Erdős–Rényi [18] random graphs $G(n, p)$, which let us vary density in a controlled way while largely avoiding the explicit community structure of the clustered benchmark. As $p$ increases, these graphs transition from sparse (often disconnected) instances with many trivial separator opportunities to well-connected instances where separator structure becomes rare and treewidth grows quickly.

**Experimental Setting.** We generate a grid of Erdős–Rényi instances $G(n, p)$ with fixed seeds, using $n$ in the range 30–80 vertices and edge probabilities $p$ in the range 1%–4%. For each parameter setting we sample five graphs and report average wall-clock runtime. Each instance is solved under all configurations and all modes. We record runtime and count runs exceeding the per-run timeout of 60 seconds as timeouts.

**Results.** To summarize behavior beyond raw $(n, p)$ parameters, we group instances by their optimal treewidth and average runtimes across modes (we only consider successful runs across all modes and configurations). Figure 3 reports the average runtime grouped by the optimal treewidth. All three configurations exhibit the same overall dependence on treewidth in this setting: runtimes remain near-zero for very small widths and then increase steeply once the treewidth reaches the moderate range, reflecting that treewidth itself is the dominant driver of difficulty for these Erdős–Rényi instances. Within this shared trend, the separator-based variants consistently improve upon the *Regular* encoding, indicating that structure-aware reasoning reduces solving time even on random graphs. It is worth noting that when treewidth increases, all solvers' runtime increases (follows similar trend), suggesting that once the instances become intrinsically hard (high width), all configurations are increasingly constrained by the same underlying SAT/MaxSAT effort.

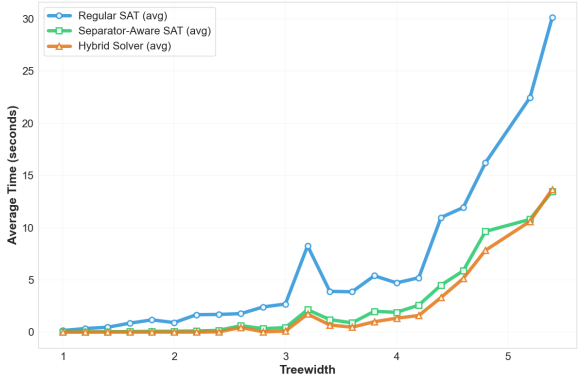


**Figure 3: Average runtime versus treewidth on Erdős–Rényi instances, averaged across modes.**

See Appendix for full grid results detailed in C.

## 7.4 SAT-level enhancements

We isolate the two SAT-layer optimizations (Section 5.5) used by HybridSolver when it can no longer split a graph and has to rely on the underlying backend (which is the *Regular* Berg–Järvisalo routine).

We therefore evaluate the effect of: (i) *bounds-awareness*, which restricts the explored widths to a certified interval $[LB, UB]$ rather than $[1, |V|-1]$, and (ii) *warm-starting*, which injects polarity preferences for the ordering variables $ord_{ij}$ (Variable 1) derived from a heuristic elimination ordering, without adding constraints.

**Bounds-aware solving.** On the `david` graph from the Berg–Järvisalo benchmark, we evaluate bounds-aware solving by running each exact mode five times with and without initial bounds. This instance has $n = 87$ vertices, so without bounds the atomic search ranges over widths $w \in \{1, \ldots, |V|-2\} = \{1, \ldots, 85\}$. With heuristic bounds enabled, the initial window is $[LB, UB] = [10, 13]$ and the true treewidth is $\mathrm{tw} = 13$. Table 1 reports mean wall-clock runtimes and resulting speedups, showing consistent improvements across all four modes, with the largest gains for `iter-lin-down` and `maxsat`.

**Table 1: Effect of initial bounds on `david`: mean runtime (s) over five runs and speedup ($LB = 10$, $UB = 13$, $\mathrm{tw} = 13$).**

| Strategy | Without bounds | With bounds | Speedup |
|---|---|---|---|
| `iter-bin` | 58.797 | 38.702 | 1.52× |
| `iter-lin-u` | 59.843 | 54.701 | 1.09× |
| `iter-lin-d` | 76.322 | 39.200 | 1.95× |
| `maxsat` | 69.464 | 41.102 | 1.69× |

Nevertheless, Figure 4 illustrates the typical runtime profile of binary search over the width: while the interval shrinks quickly at first, the expensive iterations concentrate near the optimum, where further narrowing becomes progressively harder. Bounds-aware solving therefore primarily removes avoidable work away from the optimum (as usually the heuristic bounds are quite tight), but it does not eliminate the intrinsic difficulty of proving (or refuting) widths close to the true treewidth, which is often the dominant cost in exact treewidth computation. This behavior matches the trends reported by Berg and Järvisalo [9] for iterative SAT-based treewidth solving, where runtime is likewise dominated by the final iterations near the true width.

**Warm-start hinting.** Warm-starting provides the SAT solver with an initial *hint* biased toward a promising elimination ordering—in our case derived from the greedy upper-bound ordering—with the goal of steering early branching decisions in a favorable direction. In practice, its impact is highly instance-dependent: it can yield slight speedups with high variance, but often has little to no effect. A key reason is that the hint is not a hard constraint: modern SAT solvers (e.g., Glucose) treat phase suggestions as preferences and may ignore or override them as search progresses and conflicts accumulate. We provide an illustrative example on `myciel4.gr`

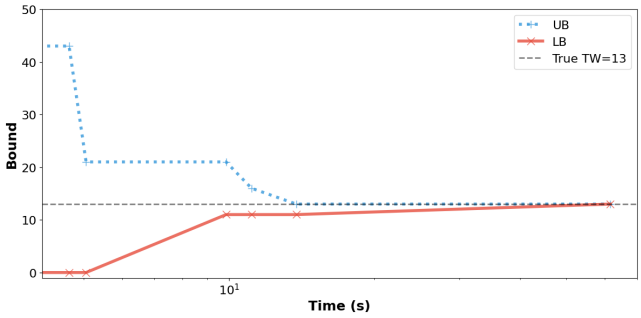


**Figure 4: Binary search bounds over time for `david.gr` without initial bounds.**

in Figure 15 (Appendix), where `iter-lin-down` improves slightly; overall, warm-starting should be viewed as a low-cost, opportunistic optimization rather than a mechanism that drastically reduces overall runtime.

## 7.5 Bound heuristics accuracy

As described in Section 5.2, HybridSolver uses upper and lower bounds on treewidth to both prune and guide exact solving. We therefore evaluate the quality of the two bound heuristics used throughout HybridSolver in isolation.

**Experimental Setting.** We evaluate bounds on 200 instances from the PACE2017 exact treewidth benchmark (*regular* and *bonus* instances), for which optimal treewidth values are available. Instance sizes range from 48 to 3,706 vertices (mean 383.9) and span diverse structures, where associated treewidth ranges from 6 to 908 with a mean value of 34.48.

**Results.** Figure 5 reports the distribution of normalized errors $e_{\mathrm{UB}}(G)/n$ and $e_{\mathrm{LB}}(G)/n$ over the PACE2017 instances. The greedy upper bound is typically close to optimal (most mass near zero), while the lower bound is looser and exhibits a longer tail. Nevertheless, both bounds remain reasonably tight relative to graph size on the majority of instances, and in a nontrivial fraction of cases they coincide, allowing HybridSolver to solve the instance without having to invoke the SAT/MaxSAT backend.

Overall, these results confirm that simple polynomial-time bounds provide strong guidance in practice, consistent with prior work on treewidth heuristics [13, 14]. Inside HybridSolver, even when bounds do not meet exactly, they substantially narrow the hard width region that the atomic SAT/MaxSAT backend must explore, improving pruning, early termination, and focus on the intrinsically difficult regime near the true treewidth. See Appendix E for more details.

## 7.6 PACE2017

We finally consider the PACE2017 exact treewidth benchmark, which serves as a test for highly specialized competitive exact treewidth solvers. We focus on the regular instances (odd-numbered: `ex001`, `ex003`, …), yielding 100 graphs with treewidths ranging from 6 to 908 (mean 37.8), and vertex counts min $|V| = 48$, max $|V| = 3706$, with an average of $\approx 413$ vertices.

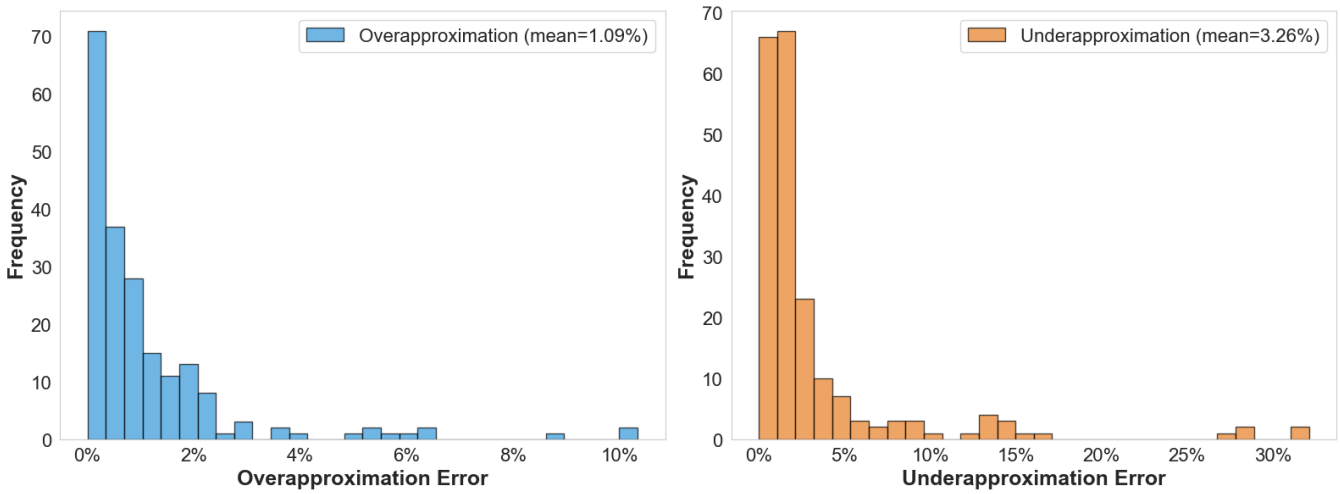


**Figure 5: Normalized error distributions of bound heuristics on PACE2017 instances. Left: upper-bound error $e_{\mathrm{UB}}(G)/n$. Right: lower-bound error $e_{\mathrm{LB}}(G)/n$.**

**Experimental setup.** We evaluate the three SAT-based configurations introduced earlier—*Regular*, *Separator-aware*, and *Hybrid-Solver*—but this time under the same mode *iter-bin* (iterative binary search over the width). Each graph instance is run once per configuration with a timeout of 5 minutes. For context, we also report results for the two top-performing PACE2017 solvers: Tamaki's positive-instance driven solver (PACE2017 1st place) [22] and Jdrasil (PACE2017 2nd place) [8], rerun under the same hardware and timeout policy.

**Results.** Among the SAT-based configurations, only HybridSolver solves any PACE2017 instance to optimality within the time limit, while the PACE2017 winning solvers solve most instances without timing out. This is expected given the goal of the benchmark: PACE2017 is designed to separate treewidth-specific engines that rely on highly domain-guided strategies from SAT-based approaches that encode the problem and delegate solving to a SAT solver. Notably, the few instances solved by HybridSolver are precisely those where it can prune instantly because the heuristic bounds coincide ($LB = UB$), certifying optimality without invoking the SAT/MaxSAT backend. Beyond these cases, once the solver must fall back to the atomic SAT backend, the remaining subproblems are typically too large, and runs time out. In most PACE2017 instances, safe-separator decomposition yields only limited and often unbalanced reductions, leaving a dominant residual leaf that remains beyond the reach of the atomic backend.

To quantify how much safe-separator decomposition reduces the instance before having to rely on our SAT/MaxSAT backend on those specific graphs from PACE2017, Figure 6 plots the size of the largest remaining atomic leaf subproblem (graph which cannot be decomposed further) versus the original graph size (points below $y = x$ indicate reduction). While many instances admit nontrivial reductions, these are often unbalanced, leaving one dominant residual subinstance close to the original size—hence the limited impact on overall solvability at PACE scale.

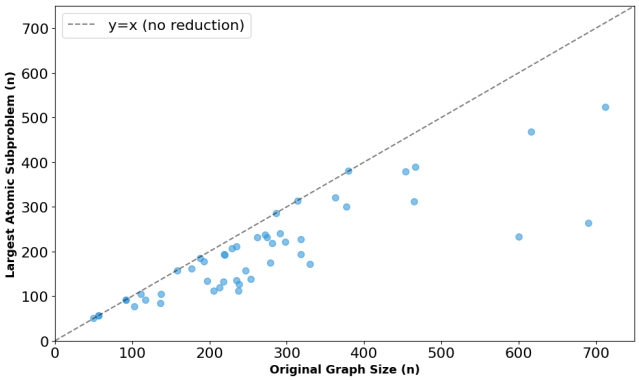


**Figure 6: PACE2017: size of the largest atomic leaf subproblem after safe-separator decomposition versus the original graph size. The dashed line $y = x$ indicates no reduction.**

## 8 Discussion and Limitations

SAT- and MaxSAT-based exact treewidth solving can benefit substantially by incorporating graph structure, but the empirical experiments also clarify where the approach remains constrained. When instances admit meaningful safe-separator decompositions, reducing the size of the hardest residual subproblem can yield orders-of-magnitude speedups, as most clearly seen on clustered graphs where both *Hybrid* and *Separator-aware* dominate the *Regular* Berg–Järvisalo solver.

Comparing the two integrations highlights a trade-off. HybridSolver uses decomposition as control flow and combines it with bounds and bounds-aware atomic solving, whereas the separator-aware approach compiles the full decomposition into one global incremental SAT/MaxSAT instance. With many leaves and repeated width checks, this global coupling can benefit from clause reuse and solver-internal pruning across leaves, and a global width-$w$

query can fail as soon as one leaf forces width $> w$, which helps explain why separator-aware can outperform HybridSolver on the hardest clustered instances.

The main limitation is that when separator structure is scarce or highly unbalanced, both decomposition-based variants increasingly face the same bottleneck as the baseline encoding. This occurs on denser random graphs and on the PACE2017 instances, where safe separators often remove small fragments while leaving one dominant residual subgraph close to the original size, thus being equivalent to having to rely on the original Berg–Järvisalo encoding, for which those instances are beyond feasibility. Consequently, while safe separators are highly effective on structured instances, the method remains limited when graphs do not decompose meaningfully.

## 9 Future Work

Beyond the improvements reported here, several other alternatives were explored but were not yet conclusive, and therefore remain open directions.

First, we ran early trials to exploit *parallelism* in the iterative SAT workflow by solving multiple width queries concurrently (e.g., testing several values of $w$ in parallel). This was not consistently beneficial: parallel execution requires separate solver instances, which forfeits incremental solving and learned-clause reuse across successive width probes, often increasing total effort.

Second, we experimented with exploiting a good heuristic elimination ordering more aggressively than warm-start phase hints. Concretely, the idea was to fix the first $k$ positions of the ordering to match a heuristic prefix by directly constraining the ordering variables $ord_{ij}$ (i.e., enforcing $ord_{uv} = 1$ for pairs $(u, v)$ whose relative order is implied by the fixed prefix), and then let the SAT solver complete the remaining suffix, effectively searching over all completions consistent with a promising start in the hope of improving the upper bound. In preliminary trials this did not yield consistent gains: fixing such $ord_{ij}$ variables can easily overconstrain the formula and push the solver into harder regions of the search space, and a prefix that is locally good for the heuristic is not necessarily compatible with an optimal global ordering. Still, this direction seems worth revisiting if one can identify conditions under which committing to a greedy prefix is provably safe.

Third, we attempted to exploit separators of unknown safeness to force additional decomposition when the graph is effectively unbreakable with respect to the safe-separator classes used in this work. For instance, we tried selecting small vertex cuts (e.g., mincut separators) and cliquifying them, treating them as if they were safe. This approach was not effective in practice and comes with two major pitfalls. The first is algorithmic: committing to an unsafe separator is hard to undo, and exploring multiple candidates with backtracking is not viable here because each downstream SAT/MaxSAT call can be expensive even on reduced instances, making branching over separator choices unfeasible in practice. The second is quality: cliquifying a separator that is not safe typically induces a substantial increase in width, often producing a loose over-approximation that is not a useful upper bound. Nevertheless, it may be possible to make progress on this direction by designing verifiable safety tests or by integrating such cuts only as optional guidance rather than as committing transformations, which remains an open direction for future work.

## 10 Conclusion

This report studied SAT- and MaxSAT-based exact treewidth computation with an emphasis on exploiting graph structure. Building on the Berg–Järvisalo encoding as an atomic backend, it integrated safe separators in two ways: a recursive meta-solver (HybridSolver) combining bounds, decomposition, warm-start and bounds-aware solving, and a separator-aware iterative encoding that compiles a full decomposition into one incremental SAT/MaxSAT instance by namespacing leaf encodings and linking their width thresholds globally.

Experiments confirm that the Berg–Järvisalo-style SAT/MaxSAT backend is a strong and competitive starting point: it provides a robust atomic solver on moderate instances and, crucially, it benefits noticeably from the proposed extensions. Across all evaluated settings, including clustered and Erdős–Rényi random graphs, explicitly leveraging graph structure within the SAT/MaxSAT framework leads to substantial gains whenever the instance decomposes well, frequently turning baseline timeouts into fast solves. The separator-aware variant can further benefit from solver reuse and cross-leaf pruning when many leaves and repeated width checks occur.

Overall, the results highlight a useful interplay between structure and generality in SAT-driven treewidth. SAT/MaxSAT encodings provide a uniform, solver-agnostic core that is easy to reuse and extend, but they benefit strongly from making graph structure explicit at the instance level.

## References


[1] [n. d.]. bnlearn Bayesian Network Repository. Online repository. https://www.bnlearn.com/bnrepository/

[2] [n. d.]. Treewidth benchmark archive (libtw.zip). Online archive. https://www.treewidth.com/docs/libtw.zip

[3] [n. d.]. TreewidthLIB graph coloring instances (coloring.zip). Online archive. https://www.staff.science.uu.nl/~bodla101/treewidthlib/coloring.zip

[4] Ignasi Abío, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. 2013. A Parametric Approach for Smaller and Better Encodings of Cardinality Constraints. In *Proceedings of the 19th International Conference on Principles and Practice of Constraint Programming (CP 2013) (Lecture Notes in Computer Science, Vol. 8124)*. Springer, 80–96. doi:10.1007/978-3-642-40627-0_10

[5] Roberto Asín, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. 2011. Cardinality Networks: A Theoretical and Empirical Study. *Constraints* 16, 2 (2011), 195–221. doi:10.1007/s10601-010-9105-0

[6] Gilles Audemard and Laurent Simon. [n. d.]. Glucose SAT Solver (official webpage). http://www.labri.fr/perso/lsimon/glucose/. Accessed 2026-01-08.

[7] Gilles Audemard and Laurent Simon. 2009. Predicting Learnt Clauses Quality in Modern SAT Solvers. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009)*. 399–404.

[8] Max Bannach, Sebastian Berndt, and Thorsten Ehlers. 2017. Jdrasil: A Modular Library for Computing Tree Decompositions. In *16th International Symposium on Experimental Algorithms (SEA 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 75)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 28:1–28:21. doi:10.4230/LIPIcs.SEA.2017.28

[9] Jeremias Berg and Matti Järvisalo. 2014. SAT-Based Approaches to Treewidth Computation: An Evaluation. In *2014 IEEE 26th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 328–335. doi:10.1109/ICTAI.2014.57

[10] Armin Biere, Marijn J. H. Heule, Hans van Maaren, and Toby Walsh (Eds.). 2009. *Handbook of Satisfiability*. Frontiers in Artificial Intelligence and Applications, Vol. 185. IOS Press.

[11] Hans L. Bodlaender. 2005. Discovering Treewidth. In *SOFSEM 2005: Theory and Practice of Computer Science (Lecture Notes in Computer Science, Vol. 3381)*. Springer, 1–16. doi:10.1007/978-3-540-30577-4_1

[12] Hans L. Bodlaender and Arie M. C. A. Koster. 2003. *Safe Separators for Treewidth*. Technical Report ZIB-Report 03–32. Konrad-Zuse-Zentrum für Informationstechnik Berlin (ZIB). Technical report.

[13] Hans L. Bodlaender and Arie M. C. A. Koster. 2010. Treewidth computations I. Upper bounds. *Information and Computation* 208, 3 (2010), 259–275. doi:10.1016/j.ic.2009.03.008

[14] Hans L. Bodlaender and Arie M. C. A. Koster. 2011. Treewidth computations II. Lower bounds. *Information and Computation* 209, 7 (2011), 1103–1119. doi:10.1016/j.ic.2011.04.003

[15] Víctor Dalmau, Phokion G. Kolaitis, and Moshe Y. Vardi. 2002. Constraint Satisfaction, Bounded Treewidth, and Finite-Variable Logics. In *Proceedings of CP (LNCS, Vol. 2470)*. Springer, 310–326.

[16] Rina Dechter. 1999. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence* 113, 1-2 (1999), 41–85. doi:10.1016/S0004-3702(99)00059-4

[17] Niklas Eén and Niklas Sörensson. 2004. An Extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003) (LNCS, Vol. 2919)*. 502–518.

[18] Paul Erdős and Alfréd Rényi. 1959. On Random Graphs I. *Publicationes Mathematicae Debrecen* 6 (1959), 290–297.

[19] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, Gaël Varoquaux, Travis Vaught, and Jarrod Millman (Eds.). Pasadena, CA, USA, 11–15. doi:10.25080/TCWV9851

[20] Chu Min Li and Felip Manyà. 2009. MaxSAT, Hard and Soft Constraints. In *Handbook of Satisfiability*, Armin Biere, Marijn J. H. Heule, Hans van Maaren, and Toby Walsh (Eds.). Frontiers in Artificial Intelligence and Applications, Vol. 185. IOS Press, 613–631.

[21] Neil Robertson and P. D. Seymour. 1986. Graph minors. II. Algorithmic aspects of tree-width. *Journal of Algorithms* 7, 3 (1986), 309–322. doi:10.1016/0196-6774(86)90023-4

[22] Hisao Tamaki. 2017. Positive-instance driven dynamic programming for treewidth. In *25th Annual European Symposium on Algorithms (ESA 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 87)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 68:1–68:13. doi:10.4230/LIPIcs.ESA.2017.68

[23] Hisao Tamaki. 2021. A heuristic for listing almost-clique minimal separators of a graph. arXiv:2108.07551 [cs.DS] https://arxiv.org/abs/2108.07551

# Appendix

## A Extended Experimental Evaluation: Berg–Järvisalo Benchmark

This appendix provides additional results and information for the the Berg–Järvisalo benchmark beyond what is presented in the main paper (Section 7.1).

### A.1 Aggregate performance metrics

Figure 7 summarizes aggregate performance statistics: average solving time over solved instances (by all configurations and modes), and the total number of timeouts.

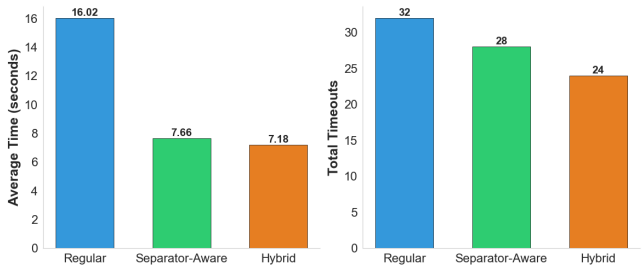


**Figure 7: Aggregate performance metrics on the Berg–Järvisalo benchmark. Left: average time on successful instances. Right: number of timeouts.**

Overall, Hybrid performs best: it solves the largest number of instances (fewest timeouts) and is also the fastest on successful runs. For the runtime comparison, we report the average over the subset of instances solved by all configurations, so times are directly comparable. *Separator-aware* comes second and already yields a clear improvement over the *Regular* baseline, confirming that exploiting safe-separator structure substantially helps in practice. Finally, *Hybrid* retains an additional advantage thanks to the early pruning capacities stemming from the upper-bound being equal to the lower-bound.

### A.2 Pairwise solver comparison

Figures 8–10 compare solving times instance-wise on a log–log scale (diagonal $y = x$). For each instance and configuration, the plotted time is the *average* over the four solving modes (`iter-bin`, `iter-lin-up`, `iter-lin-down`, `maxsat`), considering only successful runs. Points below the diagonal favor the $y$-axis configuration.

Both Hybrid and Separator-aware outperforms Regular (Figures 8 and 9). In Figure 10, points concentrate in the lower-left (instances solved quickly by both), yet Hybrid is slightly faster more often, which we attribute to earlier pruning enabled by heuristic lower/upper bounds even when separator decomposition already helps.

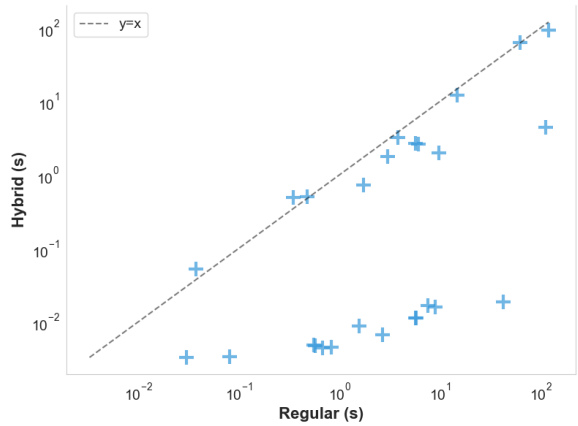


**Figure 8: *Hybrid* vs. *Regular*.**

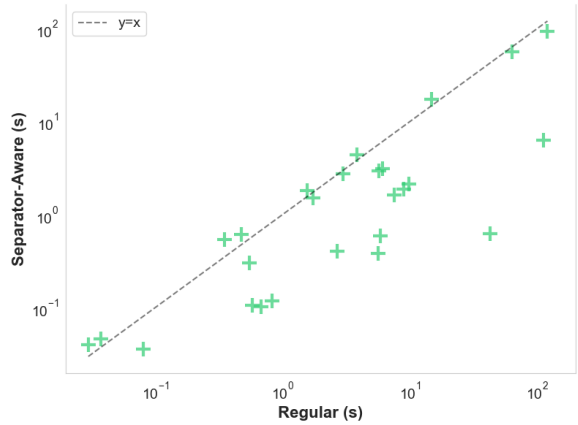


**Figure 9: *Separator-aware* vs. *Regular*.**

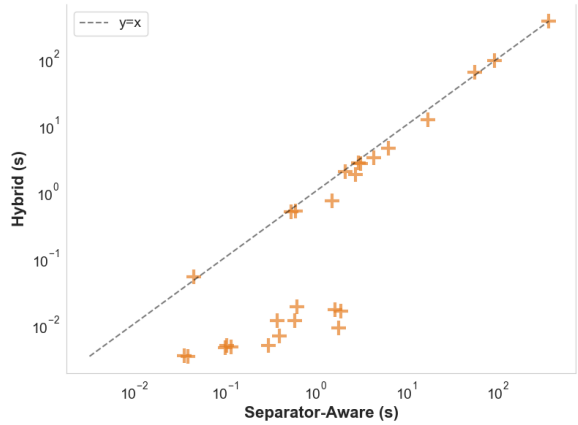


**Figure 10: *Hybrid* vs. *Separator-aware*.**

| Instance | TW | $\|V\|$ | $\|E\|$ | Regular | | | | Separator-Aware | | | | Hybrid | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | Iter-Lin-U | Iter-Lin-D | Iter-Bin | MaxSAT | Iter-Lin-U | Iter-Lin-D | Iter-Bin | MaxSAT | Iter-Lin-U | Iter-Lin-D | Iter-Bin | MaxSAT |
| alarm | 4 | 37 | 65 | 0.45 | 0.67 | 0.53 | 0.64 | **< 0.1** | 0.11 | 0.11 | 0.11 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| anna-pp | 12 | 22 | 148 | 2.51 | **0.93** | 5.06 | 3.46 | 4.30 | 1.81 | 2.79 | 1.98 | 1.35 | 3.07 | 1.35 | 1.33 |
| barley | 7 | 48 | 126 | 5.64 | 5.81 | 5.56 | 5.63 | 2.95 | 3.05 | 3.07 | 2.67 | **2.50** | 2.64 | 2.49 | 3.07 |
| barley-pp | 7 | 26 | 78 | 0.43 | **0.43** | 0.45 | 0.58 | 0.55 | 0.60 | 0.60 | 0.66 | 0.54 | 0.51 | 0.53 | 0.44 |
| barley2 | 7 | 48 | 126 | 5.67 | 6.80 | 5.65 | 6.11 | 2.79 | 3.50 | 3.17 | 2.97 | 2.24 | 3.02 | **2.23** | 2.87 |
| celar02 | 10 | 100 | 311 | 61.33 | 227.53 | 114.15 | 41.95 | 4.73 | 8.39 | 6.02 | 6.07 | 3.83 | 5.86 | **3.80** | 4.40 |
| celar09pp | 7 | 67 | 165 | 3.00 | 10.92 | 4.40 | 4.20 | 0.37 | 0.39 | 0.37 | 0.38 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| child | 3 | 20 | 30 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| david | 13 | 87 | 406 | 109.26 | 132.38 | 116.26 | 120.97 | 96.58 | **60.05** | 106.17 | 110.39 | 78.83 | 78.83 | 82.36 | 97.66 |
| david-pp | 13 | 29 | 191 | 1.92 | 2.47 | 4.54 | 6.31 | 3.95 | 3.15 | 5.86 | 4.53 | 2.30 | 5.25 | 4.09 | **1.33** |
| eil51.tsp | - | 51 | 140 | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO |
| hailfinder | 4 | 56 | 99 | 1.84 | 4.20 | 2.47 | 2.25 | 0.37 | 0.45 | 0.39 | 0.40 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| hepar2 | 6 | 70 | 158 | 3.38 | 10.68 | 4.58 | 4.65 | 0.59 | 0.60 | 0.57 | 0.60 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| huck | 10 | 74 | 301 | 4.34 | 14.82 | 5.37 | 5.59 | 1.58 | 1.60 | 1.58 | 1.70 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| jean | 9 | 77 | 254 | 4.82 | 17.59 | 7.04 | 6.27 | 1.75 | 2.03 | 1.77 | 1.97 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| mainuk | 7 | 48 | 198 | 1.16 | 1.98 | 1.57 | 1.52 | 1.60 | 2.01 | 1.78 | 1.79 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| mildew | 4 | 35 | 80 | 0.45 | 0.72 | 0.50 | 0.53 | 0.29 | 0.31 | 0.29 | 0.31 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| miles500 | - | 128 | 1170 | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO |
| mulsol.i.5-pp | 31 | 119 | 2556 | TO | TO | TO | TO | TO | TO | TO | TO | **44.80** | 227.82 | 45.11 | 147.31 |
| myciel3 | 5 | 11 | 20 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| myciel4 | 10 | 23 | 71 | 16.14 | 11.58 | 13.62 | 17.28 | 20.23 | 11.97 | 21.87 | 15.15 | **7.11** | 11.99 | 7.11 | 16.05 |
| myciel5 | - | 47 | 236 | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO |
| oesoca | 3 | 39 | 67 | 0.53 | 0.86 | 0.62 | 0.70 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| oesoca+-pp | 11 | 14 | 75 | 0.36 | **0.26** | 0.31 | 0.46 | 0.48 | 0.57 | 0.48 | 0.59 | 0.51 | 0.41 | 0.50 | 0.53 |
| oesoca+ | 11 | 67 | 208 | TO | TO | TO | TO | 453.84 | 417.94 | **270.68** | 319.46 | 436.02 | 391.96 | 292.28 | 352.75 |
| oesoca42 | 3 | 42 | 72 | 0.65 | 1.06 | 0.73 | 0.88 | 0.11 | 0.12 | 0.12 | 0.12 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| pathfinder | 6 | 109 | 211 | 13.36 | 119.24 | 19.56 | 17.66 | 0.60 | 0.65 | 0.60 | 0.63 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| pathfinder-pp | 6 | 12 | 43 | 0.03 | 0.03 | 0.03 | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** | **< 0.1** |
| queen5_5 | - | 25 | 160 | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO |
| queen6_6 | - | 36 | 290 | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO |
| queen7_7 | - | 49 | 476 | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO | TO |
| ship-ship-pp | 8 | 30 | 77 | 68.17 | **53.49** | 60.68 | 68.69 | 53.72 | 55.48 | 58.28 | 58.00 | 61.94 | 63.23 | 62.93 | 62.14 |
| water | 9 | 32 | 123 | 1.06 | 2.34 | 2.54 | 0.95 | 1.26 | 1.44 | 1.84 | 1.51 | 0.71 | 0.97 | **0.54** | 0.67 |
| win95pts | 8 | 76 | 225 | 6.02 | 16.41 | 8.99 | 7.64 | 1.94 | 2.37 | 2.05 | 2.17 | 2.09 | **1.82** | 2.09 | 1.92 |

**Table 2: Running times (seconds) on the Berg–Jarvisalo benchmark for our SAT-based configurations.**

# B Extended Experimental Evaluation: Pseudo-Random Clustered Graphs

This appendix provides additional plots and breakdowns for the pseudo-random clustered benchmark (Section 7.2), complementing the main-paper heatmap view.

## B.1 Example instance

Figure 11 illustrates a representative generated instance. Dense intra-cluster connectivity contrasts with sparse inter-cluster links routed through a small set of gateway vertices, which is precisely the type of structure that yields many small separators. Each instance is generated by a two-level random process. For given parameters $(k, n)$, the vertex set is partitioned into $k$ clusters $C_0, \ldots, C_{k-1}$ of size $n$, with

$$C_i = \{i \cdot n,\ i \cdot n + 1,\ \ldots,\ i \cdot n + (n-1)\}.$$

Within each cluster $C_i$, we sample an Erdős–Rényi graph $G(n, p_{\text{in}})$ by adding each intra-cluster edge independently with probability $p_{\text{in}}$ = `cluster_density` = 0.5. To control cross-links, we designate a single gateway vertex per cluster, $g_i := in$, and add inter-cluster edges only between gateways: for each pair $i < j$, we add the edge $\{g_i, g_j\}$ independently with probability $p_{\text{out}}$ = `inter_cluster_prob` = 0.25. No other inter-cluster edges are introduced. Overall, the generated graphs have sizes ranging from 10 to 280 vertices with associated treewidths going from 1 to 13.

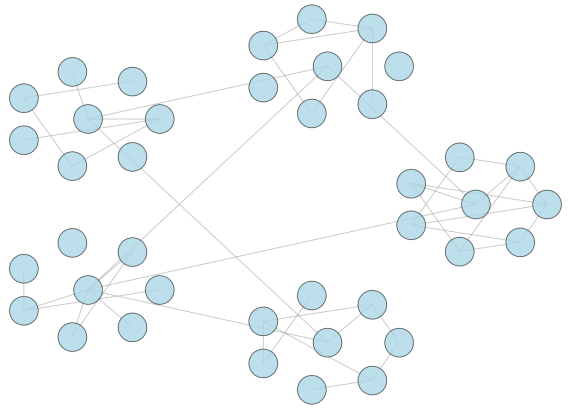

**Figure 11: Example pseudo-random clustered graph instance.**

## B.2 Scaling with cluster size (fixed $k$)

To isolate the effect of cluster size, Figure 12 fixes the number of clusters ($k = 11$) and reports runtime as $n$ increases, averaged across atomic solving modes.

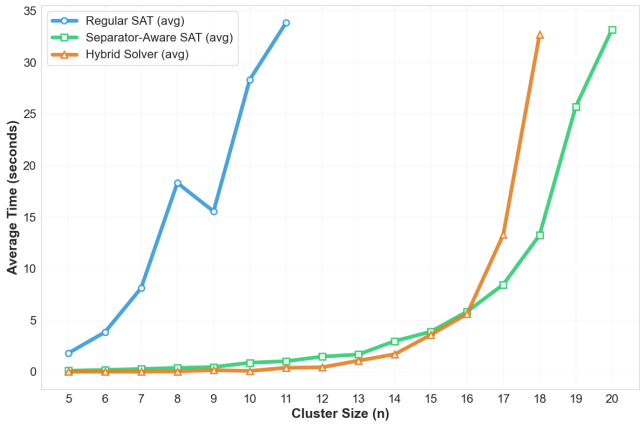


**Figure 12: Average runtime versus cluster size $n$ for a fixed number of clusters ($k = 11$), averaged across atomic solving modes.**

The baseline grows sharply and quickly reaches the hard regime. Both separator-based configurations substantially flatten this trend for small and medium cluster sizes, consistent with decomposition preventing the atomic solver from facing the full instance. We can also see that *Separator-aware* remains slightly better than *Hybrid* towards the end of the plot. Note that nevertheless, we can clearly notice the trend of exponential blowup starting from cluster size = 15 thus showing that at some point, both separtor-based enhanced solvers face unavoidably hard atomic subgraphs (the clusters).

## B.3 Timeouts

Figure 13 summarizes the total number of timeouts across solver configurations and atomic solving modes.

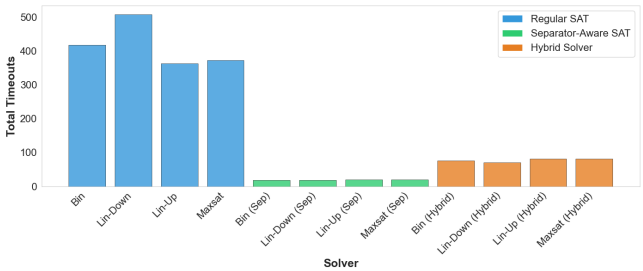


**Figure 13: Total number of timeouts per solver configuration and atomic solving mode on the pseudo-random clustered benchmark.**

The *Regular* baseline times out frequently across all modes. Both separator-based approaches substantially reduce timeouts, and *Separator-aware* drives them to near zero on this separator-rich benchmark, consistent with the cross-leaf clause reuse mechanism discussed in Section 7.2.

# C Extended Experimental Evaluation: Random Graphs

We complement the main-paper results with an extended full grid.

## C.1 More on instance generation

Each instance is sampled from the Erdős–Rényi model $G(n, p)$: meaning that for a fixed number of vertices $n$, each potential edge is included independently with probability $p$. Overall the generated graph have treewidths going from 1 to 7 (with vertices going from 30 to 80).

## C.2 Runtime landscape over $(n, p)$

Figure 14 shows the full runtime landscape over the $(n, p)$ grid on a logarithmic scale. For very small $p$, instances are often disconnected and contain many simple separators, so all configurations tend to solve quickly. As $p$ increases, the *Regular* baseline enters the hard regime earlier and accumulates timeouts slightly sooner. Both separator-based configurations delay this transition in the sparse-to-moderate density range, but the advantage narrows in the dense regime, where separator structure becomes scarce and all approaches are increasingly dominated by the underlying SAT/MaxSAT bottleneck near the optimum.

## C.3 Warm-starting via phase hints

We complement the main paper's SAT-layer study by reporting our warm-start experiment. We evaluate warm-starting on `myciel4.gr` from the Berg–Järvisalo benchmark using the `iter-lin-down` mode (descending scan). We compute a greedy upper-bound ordering and translate it into *phase/polarity hints* for precedence variables $ord_{ij}$, without adding any constraints. We run 5 descents with warm-start enabled and 5 without, recording the mean cumulative time after each tested width.

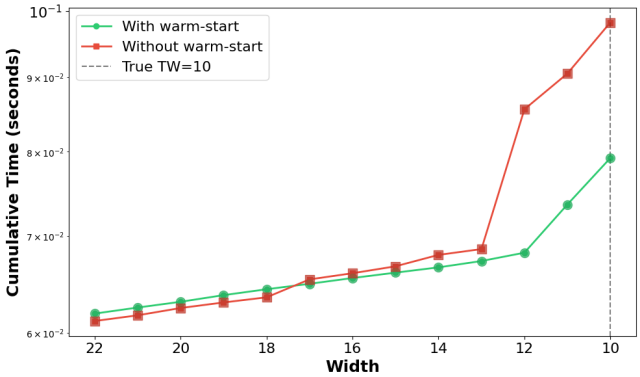


**Figure 15: Warm-start impact for `iter-lin-down` on `myciel4.gr`. Curves show mean cumulative time (5 runs) after testing each width. The dashed line marks the true treewidth (tw = 10).**

Figure 15 shows a small improvement on this instance, most visible near the optimum. More broadly, the effect is instance-dependent and usually modest.

**Table 3: Distribution of detected safe-separator types on the PACE2017 benchmark (regular instances only).**

| Separator type | Count | Proportion |
|---|---|---|
| Clique separators | 1214 | 95.4% |
| Almost-clique separators | 50 | 3.9% |
| Articulation points | 8 | 0.6% |

# D Extended Experimental Evaluation: PACE2017 (Exact Treewidth)

This appendix provides additional empirical statistics complementing Section 7.6 on the PACE2017 *regular* instances. We focus on quantifying the effect of safe-separator decomposition *prior* to invoking the atomic SAT backend.

## D.1 Reduction ratios

Figure 16 shows the distribution of reduction ratios, which is another view from the main paper to portray the limited decomposability of the PACE2017 instances. A reduction ratio of 100% means that decomposition achieves no size reduction: no safe separator is found and the largest remaining subproblem has the same size as the original graph.

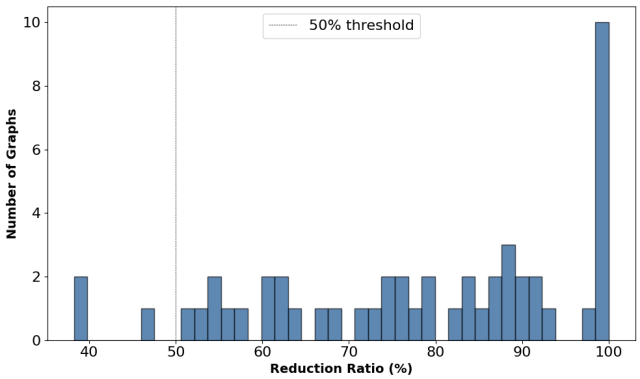


**Figure 16: PACE2017: distribution of reduction ratios induced by safe-separator decomposition. The dashed vertical line marks a 50% reduction threshold.**

## D.2 Detected safe-separator types

Table 3 summarizes which safe-separator classes are detected during recursive decomposition on PACE2017 (disconnected components omitted since all considered instances are connected).
On average, 25.44 safe separators per graph are detected across the benchmark. While this seems consequent, usually these separators are highly unbalanced (far from ideal separators which would split in half each graph instance).

# E Extended Experimental Evaluation: Bound Heuristics Accuracy

This appendix simply provides an additional statistic to Section 7.5.

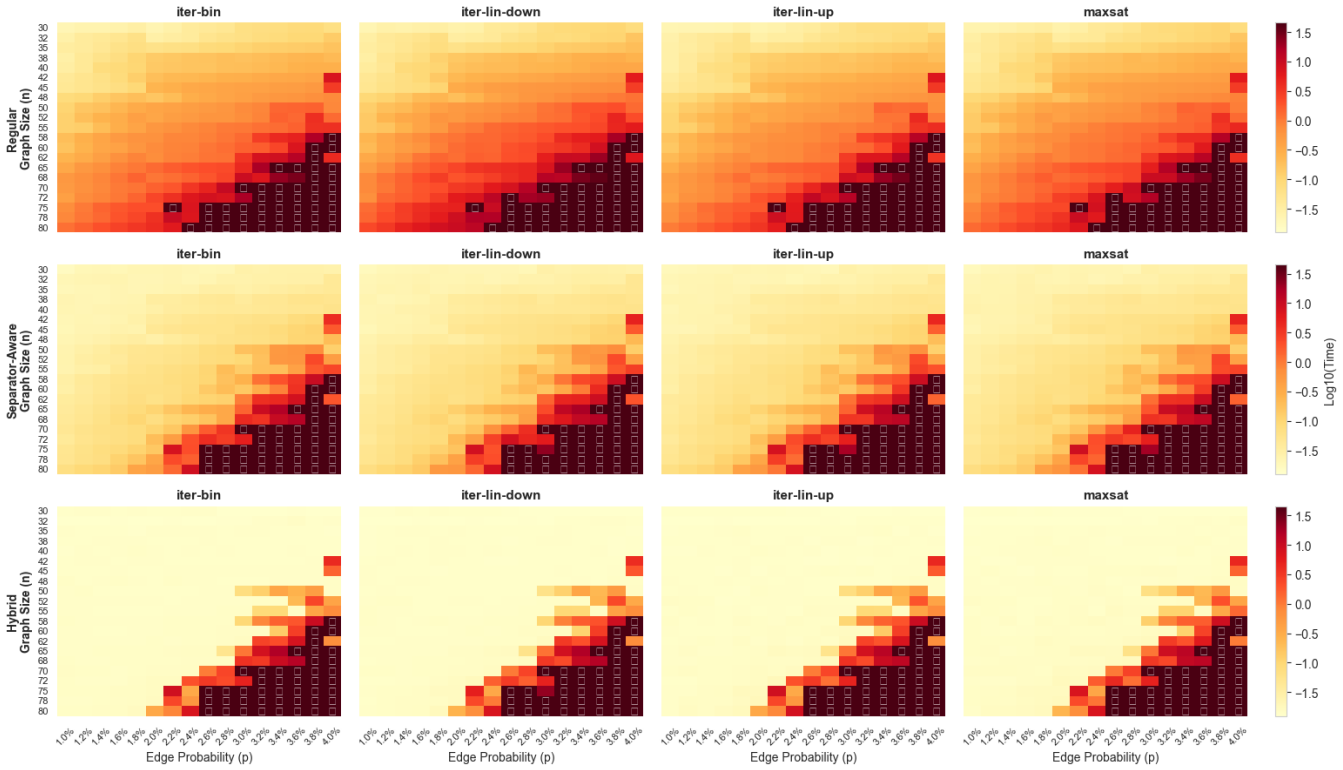


**Figure 14: Runtime heatmaps ($\log_{10}$ seconds) on Erdős–Rényi graphs over graph size $n$ and edge probability $p$, for all solver configurations and modes. White square denotes timeouts.**

## E.1 Aggregate accuracy statistics

Table 4 summarizes aggregate bound quality on PACE2017. The greedy upper bound is typically close to optimal, while MMW is looser but still provides a useful floor. The resulting gap $UB - LB$ is often small enough to substantially restrict the width interval explored by the atomic iterative SAT/MaxSAT backend, nevertheless as stated, the hardest part usually lies when close to the optimal treewidth, explaining why SAT-based solver even with enhanced upper and lower bounds timeout on the PACE2017 instances.

**Table 4: Accuracy of upper and lower bound heuristics on PACE2017 instances.**

| Metric | Upper Bound | Lower Bound |
|---|---|---|
| Mean absolute error | 2.46 | 5.93 |
| Max absolute error | 12.0 | 26.0 |
| Exact matches (%) | 21.5 | 10.0 |
| **Gap statistics** | | |
| Mean gap $UB - LB$ | 8.38 | |
| Median gap | 6 | |
| Max gap | 35 | |
| Instances with gap = 0 | 20 | |
| Instances with gap $\leq 5$ | 77 | |

# F Implementation Details

This appendix gives implementation details orthogonal to the main theoretical development, to support reproducibility and clarify core routines in pseudo-code form. The full implementation is available on GitHub: `https://github.com/NaykinYT/Treewidth`.

Additional documentation is provided in `src/README`, while experiment and testing instructions are in `benchmark/README` and `tests/README` (see also the top-level `README`).

## F.1 Detection of disconnected components

Our implementation uses NetworkX [19] to test connectivity and enumerate components, returning relabeled component subgraphs together with vertex maps old↔new for later decoding and merging.

**Complexity.** $O(|V| + |E|)$.

**Code.** `src/separator/safe/finders/disconnected.py`.

## F.2 Detection of articulation points

Our implementation relies on NetworkX [19] to enumerate articulation points. For each articulation point $a$, we form the singleton separator $S = \{a\}$, remove $S$ from the graph, and compute the connected components of $G \setminus S$ and keep track of them. Each candidate is assigned a score (using Algorithm F.10) sorted by increasing score.

**Complexity.** $O(|V|(|V| + |E|))$

**Code.** `src/separator/safe/finders/articulation.py`.

## F.3 Detection of clique separators

Clique separators are generated by enumerating cliques $S$ of size at least 2 up to a fixed limit $k_{\max}$ and testing whether removing $S$ disconnects the graph using NetworkX [19]. We keep $k_{\max}$ small in practice, typically $k_{\max} = 3$, since the number of clique candidates grows rapidly with $k_{\max}$. Enumeration proceeds by increasing clique size, so it can be stopped as soon as a clique larger than $k_{\max}$ is reached. For each candidate clique $S$, we compute the connected components of $G \setminus S$ using a subgraph view, discard candidates yielding fewer than two components, and just like for articulation nodes separator, we assign it a score using Algorithm F.10. All detected clique separators are returned sorted by increasing score.

**Complexity.** Let $n = |V|$ and $m = |E|$, and let $C_{\le k_{\max}}$ be the number of enumerated cliques of size at most $k_{\max}$. Each candidate requires a connected-components computation on $G \setminus S$, which takes $O(n+m)$ time, so the overall worst-case runtime is $O(C_{\le k_{\max}} \cdot (n+m))$. In the worst case $C_{\le k_{\max}} = O(n^{k_{\max}})$, yielding $O(n^{k_{\max}}(n+m))$ for fixed $k_{\max}$.

**Code.** `src/separator/safe/finders/clique_separator.py`.

## F.4 Detection of almost-clique separators

We follow Tamaki's $A(H, G)$ heuristic [23]: we compute a chordal completion $H$ of $G$ (via Maximum Cardinality Search (MCS) followed by triangulation), build a clique tree of $H$, and use intersections of adjacent maximal cliques as candidate separators. Each candidate $S$ is retained only if $2 \le |S| \le k_{\max}$, $S$ is a minimal separator of $G$ (i.e., $G \setminus S$ is disconnected, and no proper subset $S' \subsetneq S$ already disconnects $G$), and $S$ is an almost-clique in $G$.

**Complexity.** MCS is $O(n + m)$, while our triangulation step is $O\big(\sum_{v \in V} (d^+_\pi(v))^2\big)$ (worst-case $O(n^3)$); clique-tree construction by pairwise clique intersections is worst-case $O(c^2 n)$. Candidate validation is exponential only in $|S|$ and thus bounded by a small constant for fixed $k_{\max}$.

**Code.** `src/separator/safe/finders/almost_clique.py`.

## F.5 Decoding SAT models into tree-decompositions

This subsection explains how we reconstruct an explicit tree decomposition from a satisfying SAT assignment of the iterative Berg–Järvisalo encoding (following Bodlaender and Koster [13]). A model $M$ (result of the assignements of the SAT/MaxSAT solver) fixes (i) a total order via $ord_{ij}$ and (ii) an (oriented) triangulation via the $O_{ij}$ variables. We extract an elimination ordering $\pi$, normalize the $O$-edges to follow $\pi$, derive the later-neighbor sets $N^+(u)$, and then build bags and tree edges using the standard elimination-order construction.

**Bag construction and connectivity rule.** Given $\pi = (v_1, \ldots, v_n)$ and later-neighbor sets $N^+(u)$ (neighbors of $u$ that appear later in $\pi$ in the triangulated graph), we create one bag per vertex,

$$X_u := \{u\} \cup N^+(u).$$

To connect bags into a tree, we use the parent-by-earliest-later-neighbor rule: if $N^+(u) \neq \emptyset$, let

$$p(u) := \arg\min_{v \in N^+(u)} \mathrm{pos}(v),$$

i.e., the earliest vertex of $N^+(u)$ under $\pi$, and add the tree edge $(X_u, X_{p(u)})$. If $N^+(u) = \emptyset$, $X_u$ becomes a root bag (or is attached to a designated root, which is irrelevant for width). This attachment ensures $N^+(u) \subseteq X_{p(u)}$, and yields the running-intersection property (from Definition 1).

**Complexity.** Let $n = |V|$ and let $|O|$ be the number of extracted oriented triangulation edges. Computing pos takes $O(n)$. Extracting and normalizing positive $O_{ij}$ literals takes $O(|O|)$. Building all $N^+(u)$ takes $O(|O|)$. Constructing bags and parent pointers is $O(n + \sum_u |N^+(u)|) = O(n + |O|)$, since each oriented edge contributes once to exactly one $N^+(u)$. Thus the overall decoding runs in $O(n + |O|)$, and in the worst case $|O| = \Theta(n^2)$.

**Code.**
SAT-side decoding and $O$-edge extraction:
`src/sat_engine/encoding/iterative.py`.
Decomposition construction utilities:
`src/core/decomposition.py`.

**Algorithm 3** DecodeTD($M$) (SAT model → tree-decomposition)

**Require:** Satisfying assignment $M$ for $F_{\text{iter}}(G)$ on $n$ vertices
**Ensure:** Tree-decomposition $\mathcal{D}$ = (bags, td_edges, width)

```
π ← ExtractOrderingFromModel(M)        ▷ π = (v_1, ..., v_n)
pos[v_i] ← i for i = 1, ..., n
O ← ∅
for all positive literals O_ij in M do
    if pos[i] < pos[j] then
        O ← O ∪ {(i, j)}
    else
        O ← O ∪ {(j, i)}               ▷ normalize direction to follow π
    end if
end for
N⁺(u) ← ∅ for all u ∈ V
for all (u, v) ∈ O do
    N⁺(u) ← N⁺(u) ∪ {v}
end for
bags ← ∅,  td_edges ← [ ],  width ← 0
for i = 1 to n do                      ▷ iterate in elimination order
    u ← v_i
    X_u ← {u} ∪ N⁺(u)
    bags[u] ← X_u
    width ← max(width, |X_u| − 1)
    if N⁺(u) ≠ ∅ then
        p(u) ← arg min_{v∈N⁺(u)} pos[v]
        td_edges.append((u, p(u)))
    end if
end for
return (bags, td_edges, width)
```

## F.6 Bounds-aware iterative SAT and MaxSAT

Bounds-aware iterative SAT and MaxSAT are implemented as an extension of the baseline Berg–Järvisalo backend described in the main text (Section 3). Iterative probing strategies operate only on the current interval $[LB, UB]$, and the MaxSAT export restricts the optimization to thresholds within this interval (as described in the main paper).

**Code.** `src/sat_engine/encoding/` and `src/sat_engine/algorithm/`.

## F.7 Upper- and lower-bound heuristics

**Lower bound.** Our lower bound is the minor-min-width heuristic (MMW) from Bodlaender and Koster [14], which repeatedly contracts edges to form a sequence of minors (Definition 6). By minor-monotonicity, contracting edges cannot increase treewidth, hence any width witnessed along the contraction process yields a valid lower bound on $\mathrm{tw}(G)$. MMW maintains a working graph $H$ (initially $G$), repeatedly selects a minimum-degree vertex $v$, updates the bound with $\deg_H(v)$, and then contracts $v$ into a minimum-degree neighbor $u \in N_H(v)$ (removing $v$ and connecting $u$ to all neighbors of $v$).

**Algorithm 4** MMW lower bound

**Require:** Graph $G = (V, E)$

```
H ← G                                  ▷ working copy
LB ← 0
while V(H) ≠ ∅ do
    v ← arg min_{x∈V(H)} deg_H(x)
    LB ← max(LB, deg_H(v))
    if deg_H(v) = 0 then
        remove v from H
        continue
    end if
    u ← arg min_{x∈N_H(v)} deg_H(x)
    contract edge (u, v) in H          ▷ merge v into u
end while
return LB
```

**Upper bound: greedy elimination (min-fill / min-degree / hybrid).** For the upper bound we construct an elimination ordering $\prec$ and take its elimination width (Definition 7), which always satisfies $\mathrm{tw}(G) \le w(\prec)$. We use standard greedy elimination heuristics from Bodlaender and Koster [13] such as *min-degree* and *min-fill*, as well as a practical *hybrid* variant. All variants repeatedly eliminate a vertex $v$ from a working graph $H$: we record the set of *later neighbors* $N_H(v)$ (the neighbors still present at elimination time), update the running maximum width with $|N_H(v)|$, add fill edges to make $N_H(v)$ a clique, and finally remove $v$. The produced ordering is used both as an upper bound and as a warm-start candidate for SAT (Section F.8).

**Algorithm 5** Greedy upper bound via elimination ordering

**Require:** Graph $G = (V, E)$, method $\in$ {`min-degree`, `min-fill`, `hybrid`}

```
H ← G                                  ▷ working copy
≺ ← [ ],   later ← empty map,   UB ← 0
initialize heuristic scores (degree and optionally fill-scores)
while V(H) ≠ ∅ do
    select v ∈ V(H) according to method
    N ← N_H(v)
    append v to ≺;    later[v] ← N
    UB ← max(UB, |N|)
    for all unordered pairs {x, y} ⊆ N do
        add fill edge {x, y} to H if missing
    end for
    remove v from H
    update heuristic scores locally (for affected vertices)
end while
D ← BuildTDFromOrdering(≺, later)
return UB and {ordering = ≺, decomposition = D}
```

The difference between the heuristics lies only in how the next vertex is chosen: *min-degree* picks a minimum-degree vertex; *min-fill* picks a vertex minimizing the number of missing edges inside its neighborhood (ties broken by degree); *hybrid* evaluates fill-scores only for vertices up to a fixed degree threshold $\tau$ and otherwise falls

back to min-degree (a common practical trick to control fill-score overhead). Algorithm 5 summarizes the shared control flow.

**Complexity.** Throughout the experiments we used the min-fill heuristic for the hybrid solver. Both bounds (lower and upper bound variants) run in polynomial time.

**Code.**
`src/approximator/lowerbound/lower_bounds.py`,
`src/approximator/upperbound/upper_bound.py`,
`src/approximator/utils.py`,

## F.8 Warm-start hints to the SAT solver

Whenever the greedy upper-bound heuristic returns an elimination ordering $\prec$, we use it to *warm-start* the exact solver by providing *phase/polarity hints* rather than adding constraints. This preserves exactness: the SAT/MaxSAT instance is unchanged, and the solver remains free to flip any variable if needed. Let $\pi : V \to \{1, \ldots, n\}$ map each vertex to its position in $\prec$. For each ordering variable $ord_{ij}$ (encoding whether $v_i \prec v_j$), we set the preferred polarity to match the heuristic order:

$$ord^{\star}_{ij} \;=\; \begin{cases} 1 & \text{if } \pi(v_i) < \pi(v_j), \\ 0 & \text{otherwise.} \end{cases}$$

## F.9 Merging tree-decompositions

Both *HybridSolver* (Section 5) and the separator-aware decoding (Section 6) produce several child tree-decompositions that must be combined into a single tree-decomposition for the parent instance. At an internal decomposition step with separator $S$, each child subinstance contains $S$ (and after separator augmentation, $S$ is a clique), so the decompositions can be merged by introducing a *hub* bag containing $S$ and connecting every child decomposition to this hub, following the standard separator-based recombination of tree decompositions [13].

Concretely, given child decompositions $\mathcal{D}_1, \ldots, \mathcal{D}_r$, we build a new decomposition by (i) creating a fresh hub bag $X_{\text{hub}} := S$, (ii) copying all child bags and tree edges into a global structure while relabeling their bag identifiers to avoid collisions, and (iii) attaching each child decomposition to the hub through a bag that contains $S$ (which exists by construction). To keep hub attachments small, we choose the *smallest* such bag in each child. The width of the merged decomposition is $\max\big(\max_i \text{width}(\mathcal{D}_i),\ |S| - 1\big)$.

**Complexity.** Let $B$ and $E$ denote the total number of bags and tree edges across all child decompositions. Merging consists of copying and relabeling all bags and edges, plus adding $r$ hub-attachment edges. If we account for copying bag contents, the runtime is linear in the size of the resulting decomposition:

$$O\left(B + E + \sum_{X \in \text{bags}} |X|\right),$$

where the sum ranges over all bags $X$ in the *merged* decomposition (including the hub), and $|X|$ denotes the number of vertices in bag $X$.

**Algorithm 6** MergeDecompositions$(S, \{(V_i, \mathcal{D}_i)\}_{i=1}^{r})$

**Require:** Separator $S$; for each child $i$: vertex set $V_i$ and decomposition $\mathcal{D}_i = (\text{bags}_i, \text{edges}_i, \text{width}_i)$
**Ensure:** Merged decomposition $\mathcal{D} = (\text{bags}, \text{edges}, \text{width})$

```
bags ← { 0 ↦ S }                                  ▹ hub bag id 0
edges ← [ ]
offset ← 1,   maxChildWidth ← max_i width_i
for i = 1 to r do
    for all (b, X) ∈ bags_i do
        bags[b + offset] ← X
    end for
    for all (u, v) ∈ edges_i do
        edges.append((u + offset, v + offset))
    end for
    attach ← arg min_{b∈bags_i, S⊆bags_i[b]} |bags_i[b]|
    edges.append((0, attach + offset))
    offset ← offset + |bags_i|
end for
width ← max(maxChildWidth, |bags[0]| − 1)
return (bags, edges, width)
```

**Code.** `src/core/decomposition.py`

## F.10 Separator scoring heuristic

Several separator detectors (notably clique and almost-clique enumeration) may produce many candidates. To decide which separator to apply first, we attach to each candidate separator $S$ a simple *lexicographic score* computed from the sizes of the connected components of $G \setminus S$. Let $C_1, \ldots, C_r$ be the connected components of $G \setminus S$ and let $s_i = |C_i|$ denote their sizes. Our implementation defines

$$\text{score}(S) \;=\; \left(\max_i s_i,\ \frac{\max_i s_i}{\sum_i s_i},\ r\right),$$

and ranks separators by increasing score (lexicographic order), i.e., *lower is better*. Equivalently, we first minimize the size of the *largest* resulting component, then break ties by preferring more balanced splits (small $\max_i s_i / \sum_i s_i$), and finally break ties by preferring fewer components (smaller $r$) to reduce recursion/merging overhead.

**Rationale.** The dominant cost in a decomposition-based exact solver is typically the hardest subinstance. Thus, the primary objective is to shrink the largest child graph after splitting: if a separator splits a graph on $n$ vertices into two roughly equal parts of size $n/2$ (ignoring the small separator itself), then a conceptual "ordering search space" viewpoint illustrates the gain: a monolithic elimination-ordering search ranges over roughly $n!$ orderings, whereas solving two independent halves has cost on the order of $2 \cdot (n/2)!$. While our SAT/MaxSAT backend does not literally enumerate orderings, its worst-case behavior remains exponential in the number of vertices of the hardest subinstance, so separators that split "near the middle" are strongly preferred. The secondary ratio term makes this preference robust across separators of different sizes, and the final component-count tie-breaker avoids producing an excessive number of tiny subproblems when two candidates already yield the same largest-part size.

## F.11 Baseline Berg–Järvisalo SAT/MaxSAT encoding

The baseline Berg–Järvisalo iterative encoding and the corresponding exact solving modes are presented in the main text (Section 3); we therefore only point to the reference implementation.

**Code (encoding).**
`src/sat_engine/encoding/variables.py`,
`src/sat_engine/encoding/base.py`,
`src/sat_engine/encoding/cardinality.py`,
`src/sat_engine/encoding/iterative.py`,
`src/sat_engine/encoding/cnf.py`.

**Code (solving modes).**
`src/sat_engine/algorithms/iter_lin.py`,
`src/sat_engine/algorithms/iter_bin.py`,
`src/sat_engine/algorithms/maxsat.py`.

**Code (warm-start hints).**
`src/sat_engine/algorithms/warmstart.py`.

## F.12 HybridSolver

The HybridSolver recursion and control flow are already presented in the main text (Section 5); we therefore only point to the reference implementation.

**Code.**
`src/hybrid_engine/hybrid_solver.py`.

## F.13 Separator-aware SAT encoding

The separator-aware iterative encoding and its solver workflows are already described in the main text (Section 6); we therefore only point to the reference implementation.

**Code (encoding).**
`src/sat_engine/encoding/iterativeSeparatorAware.py`.

**Code (solving modes).**
`src/sat_engine/algorithm/iter_bin_separator_aware.py`.
`src/sat_engine/algorithm/iter_lin_separator_aware.py`.
`src/sat_engine/algorithm/maxsat_separator_aware.py`.